Check for updates

IET Blockchain

The Institution of Engineering and Technology

ORIGINAL RESEARCH OPEN ACCESS

Reinforcing Cloud Attack Tolerance on a Large Scale: Advanced Blockchain Technologies With Hierarchical Risk and Smart Contracts

Faiza Benmenzer[1,2] | Rachid Beghdad[1,2] | Alaa Eddine Khalfoune[1,2]

[1]Université de Bejaia, Faculté des Sciences Exactes, Département d'informatique, Bejaia, Algeria | [2]Université de Batna 2, Faculté des Mathématiques et d'Informatique, Laboratoire LAMIE, Batna, Algeria

Correspondence: Rachid Beghdad (rachid.beghdad@univ-bejaia.dz)

Received: 10 March 2025 | **Revised:** 10 July 2025 | **Accepted:** 15 September 2025

Funding: The authors received no specific funding for this work.

ABSTRACT

The distributed denial of service (DDoS) attacks represent a real danger on cloud computing systems, the existing security approaches have notable limitations such as centralization, lack of adaptability and scalability, and focusing on detection over mitigation. In this paper, we present a novel blockchain-based system to enhance attack tolerance of cloud systems against DDoS attacks by integrating a hierarchical risk-based cloud attack tolerance mechanism with smart contracts, The proposed approach ensures comprehensive threat detection and mitigation by estimating risks at all three layers: local, cluster, and global dynamically using Dempster–Shafer theory. Simulations validated the superiority of our approach compared to similar systems in terms of availability, attack detection rate, and packet loss metrics in large-scale cloud simulation of 10,000 nodes. The results demonstrate that our system not only improves security but also maintains service quality under high-intensity attack scenarios. Additionally, the implementation of our blockchain architecture shows significant scalability and cost-efficiency, thus, confirming its suitability for large cloud deployments. Furthermore, the reliability of the smart contracts against potential threats were confirmed by a formal security analysis using the Echidna tool.

1 | Introduction

By providing various and continuous services, the cloud computing become an essential part of the modern digital infrastructure. However, many security concerns have surfaced due to this increasing reliance, particularly about DDoS attacks. These attacks, are becoming more sophisticated and may pose a significant risk to the cloud systems availability [1]. Therefore, it is crucial to establish robust mechanisms for cloud attack tolerance to guaranty designing resilient systems that can withstand these threats [2].

Most of conventional cloud security systems uses intrusion detection systems (IDS) that base on signatures and anomaly detection. Although they may be effective against static challenges, their effectiveness degrades in case of emerging or changing threats. For example, the famous used signature-based IDS Snort failed to detect around 58% of zero-day DDoS attacks in 2023 due to its reliance on static patterns [3].

The majority of these methods concentrate on centralized detection, overlooking the distributed nature of cloud computing, leading to the easy identification of vulnerabilities during an attack. To address these limitations, AI-based (intelligence artificial) techniques, particularly deep learning, have been developed to improve attack identification; however, real-time detection still presents challenges, including high false alarm rates and long training times for deep learning models [4], which requires steady

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2025 The Author(s). *IET Blockchain* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

solutions between accuracy and timely response in dynamic environments.

Isolation-based strategies offer an alternative because they focus on containing attacks after detection. Resource and network isolation can mitigate the impact of attacks, but these strategies require significant resource allocation and may not be practical in all scenarios [5]. Formal methods employing mathematical modelling to define security policies and protocols offer a structured approach. These methods, while theoretically robust, can be complex to implement and may not fully adapt to the dynamic nature of cloud environments.

Hybrid approaches combine various methods for broader scenario coverage; however, these combined strategies still prove ineffective against sophisticated, large-scale cloud attacks.

Blockchain technology, known for its decentralized nature, is being explored to enhance security and resilience in such contexts [6]. The problem of single point of failure is avoided with blockchain by distributing data across multiple nodes. Its effectiveness in DDoS mitigation and secured storage is notable. Blockchain-based systems have proved high detection accuracy that ranges between 72.49% and 99.78% mitigating DDoS attacks by leveraging decentralized authentication and real-time attack detection mechanisms [7, 8]. However, the success of blockchain in large cloud environments depends on the consensus mechanism used. Key mechanisms like proof of work (PoW), practical Byzantine fault tolerance (PBFT), and proof of stake (PoS) face challenges in scalability, energy efficiency, and attack resistance. Notably, PoW's high energy consumption, PoS's centralization issues, and PBFT's scalability problems, along with vulnerabilities like the 51% attack, highlight the limitations concerning energy, scalability, attack tolerance, and security in large-scale cloud systems [9, 10]. This leads to the identification of a critical need: a new consensus mechanism that is not only decentralized but also risk-aware and adaptive to cloud-specific threats. Such a mechanism should be adaptable at scale, energy-efficient, and, most importantly, offer a high level of resilience against a wide range of security threats.

In addition to these weaknesses, a number of other drawbacks are apparent. Most of the current solutions are only limited to mere detection without adequate mitigation; many are based on centralized systems that may be subject to single points of failure. Many methods also struggle with adapting to evolving threats and often lack scalability and comprehensive security analysis. This highlights the need for new dynamic, hierarchical approach that integrates effective detection and mitigation, decentralization, adaptive and scalable algorithms, and thorough security analysis.

To overcome the above-mentioned challenges, this paper presents an innovative, multi-layered blockchain-based system designed to enhance attack tolerance in large-scale environments, the proposed scheme represents a significant advancement over traditional blockchain, it is capable to handle complex challenges of cloud security ensuring resilience across a wide range of attacks. Thus, we integrated the blockchain decentralization benefits with automation and adaptability abilities offered by smart contracts to address both detection and mitigation challenges.

The proposed system offers detection and mitigation through SCs to ensure a robust response to potential threats. It estimates risk at different levels including node, cluster, and global levels and adapts security measures based on real-time risk evaluations. This comprehensive approach not only detects threats but also dynamically adjusts security protocols to mitigate these threats effectively. In order to evaluate risks in a decentralized and efficient manner, the proposed system use the DST by assigning belief values to node attributes and generates normalized risk scores for each node. These scores are collected into cluster-level scores and further into a global risk score in parallel. If risk scores exceed predefined thresholds, appropriate defence mechanisms are triggered. The system dynamically regulates thresholds and security protocols based on these detailed risk assessments and records all data on the blockchain. Our enhanced blockchain ensures transparency and decentralization, while our SCs automate risk assessment, threshold adjustments, and consensus node selection.

The key contribution of this paper lies in the development of a novel blockchain framework that merges SCs with a multi-layered, hierarchical risk-based attack tolerance mechanism. Unlike existing hybrid blockchain-IDS systems, our approach introduces several distinct innovations. First, we perform risk evaluation using DST at three levels: node, cluster, and global enabling granular, scalable, and parallel threat assessment that adapts in real-time. Second, the system employs dynamic threshold regulation and adaptive weight adjustments based on evolving risk levels, improving accuracy, and responsiveness under varying attack conditions. Third, and most significantly, we propose a novel risk-aware consensus mechanism. By focusing on dynamic risk assessment and decentralized control, it overcomes traditional consensus strategies, enhancing both attack tolerance and scalability. Additionally, this consensus design improves energy efficiency and supports large-scale deployment. These integrated components collectively contribute to a resilient, decentralized, and adaptive defence framework capable of handling a broad spectrum of attack types, going beyond typical DDoS mitigation to include Sybil attacks, insider threats, data poisoning, and APTs.

Our methodology addresses scalability by evaluating system performance across a growing cloud environment from 1000 to 10,000 nodes using tailored metrics to assess attack tolerance, communication overhead, and resource efficiency. Performance evaluation is conducted in a large cloud, considering false positive rate, false negative rate, detection rate, availability, and packet loss rate. Gas consumption, transaction cost, and execution cost are also considered for practicality and efficiency. The security analysis of the smart contract of the proposed model is performed and evaluated using the Echidna security tool, focusing on key functionalities and resilience.

The rest of the paper is structured as follows: Section 2 provides an overview of related work. Section 3 introduces the concepts used in the work. Section 4 presents the proposed system. Section 5 includes a quantitative analysis. Finally, Section 6 outlines the conclusion and future directions for the research.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

2 | Related Works

Various attack tolerance mechanisms have been proposed in the literature to address security issues in cloud environments, particularly those targeting servers. This review covers current and cloud-relevant insights from recent research and varied techniques. This literature review section is organized into five subsections: begin with IDS-based approaches, employing both signature-based and anomaly-based intrusion detection systems. Follow with AI-based approaches, emphasizing how deep learning and other advanced AI techniques have been developed to overcome some of the limitations of traditional IDS, such as detecting unknown attacks and improving accuracy. Next, discuss isolation-based approaches that provide solutions to the limitations of detection by preventing the spread of attacks once they are detected, showcasing how resource and network isolation can mitigate the impact of attacks that bypass detection mechanisms. Then, explore formal methods that use mathematical modelling. Conclude with hybrid approaches, which combine the significant strengths of other approaches and bridge the gaps that individual methods have.

✓ IDS-based approaches

Hezavehi et al. [11] introduced an interactive anomaly-based DDoS attack detection method in cloud computing, integrating a third-party auditor (TPA) with datacentres to enhance security. The method uses a threshold anomaly-based detection strategy and simulations for validation. It improves attack detection and reduces filtering stages. However, this method faces challenges in detecting low-rate DDoS attacks, lacks adaptability to new attack types, and sets fixed resource limits without proper evaluation. The study also lacks comprehensive testing against various attack types, and the claim of the system being host-independent remains unverified.

David et al. [12] proposed a dynamic threshold and entropy measurement statistical method for effectively detecting flooding DDoS attacks in network traffic. They applied this method to a virtual network using both DARPA and CAIDA datasets, and it detected packets with high efficiency and low false positives. However, the study has some limitations. It does not detect application-layer DoS attacks, lacks techniques for identifying or mitigating the attack flows, and its adaptability and scalability are still undefined.

Tsobdjou et al. [13] proposed a method to detect flooded DDoS attacks using dynamic thresholds based on packet characteristics. The study targets (Internet control message protocol) ICMP and SYN flood attacks, using various performance metrics for evaluation. The system was tested through the GNS3 simulator in combination with the CICIDS2017 dataset and returned promising results in both real and simulated attack scenarios. However, this approach faced challenges detecting low-DDoS attacks; moreover, it needs special fine-tuning of some parameters for optimal performance.

Kautish et al. [14] introduced a scattered denial-of-service mitigation tree architecture, which aims to detect and mitigate DDoS attacks in hybrid cloud environments. This framework consists of three phases: filtering, attack detection, and attack mitigation. The proposed model was tested on a hybrid-cloud environment that combines Amazon Web Service EC2 with a simulated private cloud, using the KDD-99 dataset for evaluation. While the system shows high efficiency compared to current models, it has limitations in terms of addressing dataset shift issues. Furthermore, it does not investigate the architecture's practicality in cloud environments or its compatibility with other defence mechanisms.

Elmasry et al. [15] introduced an advanced cloud-based strategy to safeguard cloud environments from numerous malicious threats. It is composed of five main modules: monitoring, processing, analysis, prediction, and response. For added protection, a third-party intrusion detection system is employed. The system combines signature-based methods and anomaly detection techniques. Its performance surpasses empirical test results on NSL-KDD and CICIDS2017 datasets. However, the system has some limitations such as possible misclassification by outdated signature-based rules and computational resource requirements.

Vashishtha et al. [16] proposed a hybrid intrusion detection model for cloud systems. The model combines signature-based and anomaly-based models. It outperformed existing techniques in the tests that were done using three datasets. Nevertheless, it has limitations, like difficulty of scalability and adaptability to different cloud architectures, not relevant to new types of attacks, and it lacks realistic validation and analysis of false positives and false negatives.

Maheswari et al. [17] proposed an effective IDS to secure web and cloud computing. The system uses multi-verse and chaotic atom search optimization for data pre-processing, and uses chaotic manta-ray foraging optimization for clustering. It also incorporates modified teacher learning for attack labelling. The system outperforms previous studies using KDD Cup'99 and NSL-KDD datasets. However, the authors did not examine the system's scalability or attack resistance.

Elbakri et al. [18] presented a novel framework utilizing a support vector machine the pruned exact linear time algorithm to enhanced accuracy and efficiency. It is formed of four key components: feature selection, surveillance, training set, and testing set. A hybrid harmony search algorithm and the symmetrical uncertainty filter are used in the framework to distinguish between normal and abnormal network traffic in the cloud environment. The proposed scheme identifies change points within network traffic data, enabling the identification of anomalies and potential security threats with improved precision. The system has been assessed using the NSL-KDD and ISOT-CID datasets in terms of accuracy, precision, and recall where it outperforms existing detection techniques.

✓ AI-based approaches

Kushwah et al. [19] described a DDoS detection solution for cloud computing. They used a modified self-adaptive evolutionary extreme learning machine, SaE-ELM, with cross-over adaptation. This improved model automatically modifies its mutation approach and other parameters and is tested on four prominent datasets, including NSL-KDD and CICIDS 2017. Performance

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

measurements show that the system outperforms the original SaE-ELM and other machine learning models such as artificial neural networks (ANN) and support vector machines (SVM) in terms of detection efficiency. However, the model requires longer training times, posing a limitation for scenarios requiring frequent retraining.

A software defined networking (SDN)-based DDoS detection and mitigation architecture was introduced by Ribeiro et al. [20]. It protected the primary server by rerouting hostile floods to low-capacity servers using the moving target defence technique. The sensor classified flows using machine learning methods. The stacking classifier algorithm was found to be particularly effective in rapid attack detection, with the random forest algorithm achieving maximum precision. However, the study lacked variability in attack scenarios, making it less adaptable to different cloud environments.

Pasha et al. [21] presented a hybrid mechanism for low-rate AI-based DDoS. In the proposed architecture, deep learning approaches comprising sparse autoencoders and CNNs are benchmarked with traditional techniques comprising K-means and incremental SVMs. In simulated network conditions, the performance of the proposed system was evaluated based on packet loss ratio. However, this test was performed only within an attack-scale scenario, including 10 attackers. It therefore requires further tests to determine the level of its effectiveness when operating within reality in a cloud environment.

A multilayer approach to identifying and preventing DDoS assaults in cloud systems was carried out by Arunkumar et al. [22]. Traffic is captured and extracted, then grouped using K-means clustering, and attacks are identified using rule-based techniques. Supervised and unsupervised machine learning techniques are combined in this method. Using the KDD 99 dataset, the system was evaluated on a virtual cloud network and showed excellent accuracy and low false-positive rates for ICMP, transmission control protocol (TCP), and user datagram protocol (UDP) traffic. However, further scalability testing must be done in real-world cloud environments.

Addressing economic denial of service (EDoS) attacks, Suguna et al. [23] introduced a detection methodology based on stack deep polynomial network techniques. The methodology consists of multiple key components, including a dynamic threshold and continuous monitoring system. In its operational phase, the system constantly scrutinizes multivariate time-series data to identify anomalous traffic patterns indicative of EDoS attacks. While generally effective, the methodology has limitations, particularly in detecting YoYo and Slowloris attacks.

Shukla et al. [24] proposed a DDoS detection technique for cloud environments that operates at the hypervisor level. The approach combines particle swarm optimization and the radial basis function to achieve accurate identification of DDoS attacks. The system is evaluated using the NSL-KDD and KDD CUP99 datasets, which are not specifically designed for cloud-based flood attacks. The study focuses on performance metrics for UDP flood, normal, and TCP SYN flood attacks. However, the paper doesn't discuss the method's adaptability or ability to scale. The lack of a cloud-based flood assault dataset casts doubt on its efficacy in real-world cloud environments.

Mudassir et al. [25] emphasized the importance of protecting Industrial Internet of Things devices. They introduced an advanced deep learning model that classifies malicious packets and botnet attacks based on ANN, gated recurrent units (GRU), and long short-term memory (LSTM) models. According to the tests that were done using a Bot-IoT dataset, these models classify malware data accurately with good precision and recall. However, the complexity and resource-intensive nature of the model make it difficult to be used in resource-constrained IoT environments.

AlSaleh et al. [26] introduced an innovative method to detect cloud computing DDoS attacks, by combining convolutional neural networks with Bayesian machine learning to design a new model called BaysCNN, the latter helps managing uncertainties and understanding complex network traffic patterns. BaysCNN includes several steps: merging data sources, preprocessing, and using principal component analysis for feature extraction. The authors targeted issues such as class imbalance and cleaned the data by removing irrelevant features. The BaysCNN model was tested using the CICDDOS2019 dataset. The outcomes show a notable accuracy rate; nevertheless, the proposed method is complicated, highly dependent to dataset, focuses only specific attacks, and there is lack of realistic testing.

To address the challenges posed by low-rate DDoS attacks in cloud computing networks, Yin et al. [27] introduced a model called MSCBL-AND to ameliorate detection accuracy and time efficiency. The model uses convolutional neural networks (CNNs) for spatial feature extraction, bidirectional long short-term memory (Bi-LSTM) for temporal analysis, an arbitration network for feature importance prioritization, and a 2-block dense connection network for classification. Experimental results on the ISCX-2016-SlowDos dataset demonstrated that MSCBL-ADN significantly outperforms existing methods in detection accuracy and time performance. However, the model emphases exclusively on low-rate DDoS assaults, and needed requirements for validation across various datasets.

Kauhsik et al. [28] proposed an intelligent intrusion detection and prevention mechanism to improve the security of IoT environments; it merges machine learning and deep learning techniques by integrating detection and prevention functionalities to protect IoT devices from unauthorized activities. The approach is designed with scalability and can be used across large-scale IoT datasets and leverages an intrusion detection and prevention system topology to check and detect network intrusions efficiently.

Addressing the challenge of designing an accurate detection system in resource-constrained IoT environments, Nandanwar and Katarya [29] presented a hybrid deep learning model based on CNN and Bi-LSTM named TL-BILSTM, supported by transfer learning. This model contains attention mechanisms to point on critical temporal features and employs transfer learning decreasing dependence on large labelled datasets. TL-BILSTM was assessed using a benchmark dataset where it demonstrated low error rates and high accuracy, outperforming existing

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

solutions. Particularly it is more effective with imbalanced data and heterogeneous devices, which suits real-world IoT deployments.

Extending their research to Industrial IoT systems, the same authors introduced AttackNet [30], a developed intrusion detection system to deal with more dynamic and diverse environments, the authors combined gated recurrent units with CNN, AttackNet captures effectively spatial and temporal patterns of multi-variant botnet attacks. It showed notable performance in experimental evaluations, showcasing its robustness in identifying sophisticated attack patterns. This work is considered a notable step towards securing critical IIoT infrastructures by ensuring reliable anomaly detection.

✓ Isolation-based approaches

Kumar et al. [31] proposed resource isolation techniques to address service unavailability and resource contention during DDoS attacks. These techniques operate at the physical machine, virtual machine, container, and page levels to reserve or isolate resources for authenticated users. The study's experiments showed that isolating resources at the virtual machine (VM) level is most effective in reducing response time and failed requests. However, the strategy is limited against low-rate DDoS assaults and some varieties that use Internet protocol (IP) spoofing, as well as requiring more physical resources. Further research is required to investigate the scalability and practical implementation of these separation strategies.

Gurjar et al. [32] focused on mitigating amplification and reflection in DDoS attacks. They introduced three strategies: separating resources at the container level, combining container-level separation with packet limiting, and using separate network hardware. These methods aim to improve the performance of victim web services by isolating resources. Experimental results show significant improvements in test and response times. However, each approach has its drawbacks; for example, packet limiting at the container level could lead to the loss of important UDP packets. While the study is comprehensive, it lacks discussion on real-world deployment challenges and the effectiveness of these methods against other types of DDoS attacks.

Akbarian et al. [33] proposed a framework for detecting and mitigating cyberattacks in cloud control systems. They evaluated the framework using a real-world testbed and chaos mesh for network delay simulation. The study found that an observer-based method was more effective in detecting attacks quickly compared to an ARR-based method. The authors also introduced a new isolation technique that improved upon the limitations of existing ARR-based methods in terms of robustness and accuracy. However, the study did not fully explore the framework's adaptability and scalability.

Kumar et al. [34] conducted an experiment to determine the optimal timing of resource isolation as a mitigation strategy for DDoS attacks in cloud systems. The study examined five timing techniques and four categories of resource isolation for comparison. The researchers found that the "always ON" technique proved to be highly effective in reducing response time and minimizing request failures. Although the technique is highly reliable, the study suggested conducting further research to investigate its resource overhead.

Kumar et al. [35] proposed an approach to minimize the effect of DDoS attacks on victims' service in clouds. Their solution used container-based request separation and adopted the scale-inside-out strategy for suspicious requests. The study demonstrated the potential to reduce response time for legitimate users during a DDoS attack compared to current solutions. However, it has some limitations; it fails to consider adaptability or scalability concerns. Additionally, the simulations used in this study lack realistic testing data.

Patidar et al. [36] examined multiple strategies for countering DDoS attacks in cloud settings, using metrics such as test completion time, response times, and the number of failed requests. Various strategies were tested at different levels of attack concurrency, including combinations of page separation, resource allocation to white-listed users, and DDoS mitigation techniques. The study found that this combination performed best at higher concurrency levels. However, the paper didn't delve into the effects of undetected attacks, leaving an open question on how to serve white-listed users in those scenarios.

Kumar et al. [37] proposed a dual-layer defence system that integrates request filtering and containerization to mitigate DDoS effects. The first layer filters requests based on connection volume and forwards them to the second layer where bandwidth is restricted for intermediate machines. A load balancer directs traffic to various containers, serves benign users and maintaining near-normal response times. The system maintains service availability up to 98% of benign requests even under severe DDoS attacks. However, it requires additional machines, less effective against low-rate attacks below the threshold, and bandwidth restrictions may increase response time.

✓ Formal approaches

Hasan et al. [38] proposed CAMP game, a game-theoretic model for addressing co-resident attacks in cloud computing. The model uses Nash equilibrium to capture interactions between the virtual machine manager and VM, considering them as potential attackers or benign entities. The effectiveness of the model is demonstrated through CloudSim simulations, considering factors such as the number of virtual machines and defence preferences. However, the study is limited to standard cloud servers and does not explore adaptability to different architectures or compatibility with other defence mechanisms. Real-world testing for scalability is also lacking.

Liu et al. [39] introduced a framework for evaluating the survivability and vulnerability of cloud redundant array of independent disks (RAID) storage systems. Using the continuous-time Markov chain and multi-valued decision diagram, the framework assessed both disk- and system-level behaviour. While the framework effectively evaluates the survivability and vulnerability of cloud RAID storage systems, the authors do not delve into more comprehensive models. It also focuses mainly on disc and system-level behaviour, potentially overlooking other factors that could affect cloud storage survivability.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Zhou et al. [40] proposed a framework for enhancing attack tolerance in cloud-based medical information storage using coloured petri nets (CPN) for modelling. The framework addressed security vulnerabilities such as blocking, leakage, stealing, and injection. The authors extended a simpler model to include multiple users and replaced fault tolerance mechanisms with an attack tolerance framework. They used CPN Tools for simulation and conducted a case study to validate the framework's effectiveness in detecting and mitigating complex attacks. However, the study has limitations, such as the manual deployment of detectors and tolerance solutions, which are currently only applicable at the design stage.

Ouffoué et al. [41] developed a risk-based monitoring framework for securing cloud-based applications, focusing on web services and choreographed microservices. The framework uses the Mont image monitoring tool and the SChorA conformance tool to detect security incidents in real-time and perform formal verification. The effectiveness of the framework is validated through a case study on an electronic voting system, where it successfully identifies and counters insider attacks and unauthorized code changes. However, the framework is limited to programming languages that support advanced reflection application programming interfaces.

Tavori et al. [42] conducted a study on optimizing attack strategies on distributed resource systems, specifically targeting capacity and DDoS flow attacks. They used M/M/1 and M/M/1/K queuing models to determine optimal attacks under a variety of conditions, including migration costs and system parameters. The study highlights the importance of queuing models and delay metrics in shaping the optimal attack strategy and proposes algorithms to identify these strategies. However, the study is primarily theoretical and lacks real-world data for validation. The focus on specific queuing models may also limit the findings' applicability to other types of distributed systems.

Welsh et al. [43] proposed a new method for evaluating resilience in decentralized service-oriented architectures using a bio-inspired embryonic fog service delivery platform. They introduced assortativity as a metric for assessing service stability over time and found a strong correlation between assortativity and system performance. However, the study did not investigate the adaptability or scalability of the embryonic platform, and the focus on assortativity may not fully capture the complexity of resilience in these systems.

✓ Hybrid approaches

Kumar et al. [44] proposed a genetic deep belief network model driven by Dragonfly for securing virtual machines in the cloud from malware. The model consists of four stages: data collection and pre-processing, feature extraction, attack detection and classification, and cluster-based authentication. The model demonstrates good performance in terms of reconfiguration time and data overhead. However, the study highlights the fact that the attack detection process is time-consuming. Additionally, the paper does not investigate the adaptability or scalability of the model, raising concerns about its practical applicability.

GSR et al. [45] suggested a deep learning solution for recognizing the DDoS attacks on cloud environments. The framework merges attributes of convolutional neural networks and variational autoencoders to enhance detection. It demonstrates a higher detection rate and fewer false positives than traditional approaches such as decision trees and random forests. However, the study has limitations, such as using datasets that may not fully represent real-world scenarios. The framework's scalability and real-time applicability are untested, and its computational complexity could introduce overhead. The study also doesn't address deployment challenges.

Krishnan et al. [46] presented OpenStackDP, a security framework designed for SDN-managed OpenStack cloud infrastructures. The framework utilizes SDN, network function virtualization, and machine learning to enhance the security of the data plane and networking fabric. The authors demonstrated that OpenStackDP outperforms five other security solutions in terms of detection rate, latency, and central processing unit (CPU) utilization. However, the framework does have limitations in terms of data generalization, offline training, and addressing zero-day vulnerabilities and internal threats.

Shah et al. [47] proposed a model called EDOS-TSM to address TCP SYN flooding attacks in cloud environments. The model utilizes SDN and a POX SDN controller to differentiate between attacks and legitimate flows using parameters such as payload size and time to live. The model demonstrates superior performance in terms of false negatives and resource utilization. However, it has not been tested in realistic, enterprise-level environments.

Manoj [48] proposed an optimal load distribution framework for detecting DDoS attacks in cloud environments. Using mechanisms such as bat algorithms and fuzzy extreme learning machine classifiers, the study aims to improve cloud security. The framework is tested on CloudSim and uses metrics like attack detection rate and resource usage for evaluation. While the proposed classifier effectively reduces resource usage under DDoS conditions, the study has several limitations. It doesn't address adaptability or scalability, and it relies on a dataset that may not cover all types of DDoS attacks. The study is also simulation-based, lacking real-world testing, and focuses solely on VM-related DDoS attacks.

Hei et al. [49] proposed a cloud intrusion detection system that combines blockchain and federated learning, using hyperledger fabric for enhanced data security in cloud computing. The system outperformed Ethereum in terms of data upload and retrieval efficiency, as demonstrated through experiments using the KDDCup99 dataset. It focused on robust intrusion detection, privacy protection, computational offloading, and efficient feedback mechanisms. However, the study lacked real-world testing and faced challenges in managing bandwidth due to false alarms.

Using deep learning techniques, Ouhssini et al. [50] developed a real-time detection framework called DeepDefend to prevent cloud environments against DDoS attacks. The framework bases specifically on CNN-LSTM-transformer networks to forecast network traffic entropy and identify potential attacks. A genetic algorithm is used for feature selection. The system has been tested

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

on the CIDDS-001 dataset, and showed high accuracy in entropy prediction and DDoS attack detection. However, its effectiveness against a wide range of attack types and scales needs deeper tests with more realistic scenarios.

Pandithurai et al. [51] introduced a new method for predicting DDoS attacks, they relied on honey badger optimization for feature selection along with a Bi-LSTM classifier. The authors demonstrated the superiority of this model over traditional techniques, achieving higher accuracy on public datasets such as CICIDS2018-AWS, CICIDS2017, and CICDoS2016. This highlights the importance of advanced preprocessing and optimization methods improving detection performance. However, factors such as detection speed and false positives need additional research to improve.

Songa et al. [52] introduced an integrated SDN framework for early detection of DDoS traffic patterns within SDN-based cloud environments. They employed machine learning methods for data preprocessing, recursive feature elimination for dynamic DDoS attack patterns, density-based spatial clustering for large traffic data, and several time series techniques, including auto-regressive integrated moving average, Lyapunov exponent, exponential smoothing filter, dynamic threshold, and a rule-based classifier to predict chaotic behaviour. They assessed their model on the CICDDoS 2019 dataset where it achieved better accuracy levels and faster detection times, outperforming existing methods. Nevertheless, the system lacks real-world tests in different scenarios.

Blockchain technology was thoroughly examined by Nandanwar and Katarya [53] in order to develop a decentralized and intelligent security system. Their study looked at the decentralized peer-to-peer architecture of blockchain and its core security principles: authorization, confidentiality, integrity, and authentication. It also identified common weaknesses, real attack paths, and existing mitigation strategies. The evaluation also emphasized the ability of combining blockchain with state-of-the-art technologies such as internet of things and deep learning, particularly to increase the effectiveness of decentralized intrusion detection systems.

Based on concluded results, the authors subsequently created an explainable hybrid deep learning model especially intended for safeguarding Industry 5.0 cyber-physical systems. In their 2025 study, they introduced Cyber-Sentinet, an intrusion detection system that enhances model interpretability and transparency by combining Shapley additive explanations with a residual neural network [54].When Cyber-Sentinet was evaluated on the Edge-IIoT-2022 dataset, which contains a variety of sophisticated attacks like DDoS, SQL injection, and MITM, it showed good performance represented including factors like accuracy, precision, and recall. Because of its interpretable outputs, the model is particularly well-suited for dynamic, hyper-connected CPS scenarios where trustworthy threat detection and human–machine trust are essential.

Table 1 summarizes this section by comparing our approach with related works. As comparison criteria, we consider the method employed, the techniques used for attack tolerance, and whether the approach is decentralized. We also highlight adaptability as the method's flexibility in responding to diverse attack scenarios, as well as scalability as its ability to scale to cloud demands. Furthermore, the table takes into account whether the security is analysed, advantages, and any identified limitations of each method.

A critical analysis reveals several trends and gaps in current research that necessitate further discussion.

- Some approaches prioritize detection rather than mitigation; this highlights the need for integrated systems that can detect and successfully resist under threats. We addressed this gap by integrating a multi-layered, blockchain-based risk assessment strategy. This integration enables both detection and proactive mitigation of attacks, providing a more holistic approach to cloud security.
- The majority of methods focus on centralized detection, neglecting the distributed nature of cloud environments. Unfortunately, centralized systems may suffer from single points of failure and lack resilience during attacks. To address this issue, we propose decentralized architecture, with enhanced blockchain technology and an innovative consensus algorithm, the risk assessment and mitigation processes are distributed across multiple nodes, thus, central points of failure are avoided and resilience against attacks is increased.
- Many approaches fail to address their capacity for adapting against evolving attack vectors, leaving a gap in their long-term robustness. While some systems address this, they still insufficient for cloud complexity. Our proposed system addresses this limitation by incorporating adaptive algorithms and dynamic risk assessments that evolve with emerging threats. This enables dynamic adjustment to different attack conditions and tolerates diverse attack vectors, enhancing cloud resilience.
- The scalability of these methods is often not addressed, posing concerns about their suitability for large cloud environments. Our system overcomes this limitation by employing dynamic and multilayered defence mechanisms over nodes, clusters, and globally, ensuring that the system will remain efficient when the cloud scales up. Moreover, the enhanced blockchain architecture also improves scalability by enabling faster consensus and real-time, security adjustments.
- Lack of comprehensive security analysis in most of existing schemes. To address this, we conducted a security analysis of a proposed smart contract using Echidna tool and validated its effectiveness for managing security requirements.

Our approach aims to address these weaknesses and provide a more robust and practical solution for cloud security. It integrates a multi-layered, blockchain-based risk assessment strategy to detect and mitigate attacks in large cloud environments.

3 | Theoretical Reference

In this section, we introduce the concepts used in this paper.

- **DDoS attack**

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Related works comparison.

Works	Tolerance techniques	Decentralized?	Adaptability?	Scalability?	Security analysis?	Advantages	Drawbacks
[11]	Filtering method	×	✓	~	×	Improved attack detection, reduced filtering	Struggles with low-rate DDoS, high false negative
[12]	Threshold-based algorithm	×	×	×	×	Enhanced detection accuracy, low processing time	Ignore application-layer DDoS, no mitigation strategy
[13]	Information entropy, dynamic thresholding	×	×	×	×	Threshold adaptation	Need fine-tuning parameters, struggles with low-rate DDoS
[14]	Graph anomaly detection, Firewall	✓	×	×	×	Reduces threats, quick DDoS detection	Dataset shift issues, lacks validation in diverse scenarios
[15]							
[16]	Signature and anomaly-based detection	×	×	×	×	Identify known/unknown attacks	Misclassification risk, high computational overhead, poor false positives/negatives analysis
[17]	Clustering and attack classification	×	×	×	×	Improved clustering and detection accuracy	Lacks false positive/negative analysis
[18]	Hybrid harmony search, symmetrical uncertainty filter	×	×	×	×	Improved accuracy, change point detection	Increased false positive
[19]	Detection-focused approach	×	×	×	×	High detection accuracy	Extended training time, non-specific attack detection
[20]	Moving target defence	×	×	×	×	Fast attack detection	High computational overhead
[21]	Sparse autoencoder, CNN	×	×	×	×	Effectiveness of rigorous testing	Unrealistic evaluation
[22]	Traffic capture and extraction, grouping	×	×	×	×	Improved detection rates with minimal false alarms	Unrealistic evaluation
[23]	Stack deep polynomial network	×	×	×	×	Detect EDoS outliers	Challenges detecting YoYo and Slowloris attacks
[24]	Swarm optimization *i*th radial basis function	×	×	×	×	Enhanced classification accuracy	Lacks cloud-specific dataset
[25]	Data processing and botnet classification	×	×	×	×	High accuracy, learn from large datasets	Complexity and resource-intensive
[26]	Combine Bayesian methods with CNNs	×	×	×	×	High accuracy, Improved performance with data fusion	Dataset dependency, Model complexity, lacks realistic testing

(Continues)

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | (Continued)

Works	Tolerance techniques	Decentralized?	Adaptability?	Scalability?	Security analysis?	Advantages	Drawbacks
[27]	CNN and BI-LSTM	×	×	×	×	Improved accuracy	Focuses only on LDDoS, requires multi-dataset validation
[31]	Block source IP	×	~	×	×	Reduced request failures, improved response times	Ineffective against low-rate DDoS requires fine-grain separations study
[32]	Resource separation	×	×	×	×	Near-no-attack performance	Requires additional hardware resources Potential loss of important UDP packets
[33]	ARR-based methods	×	x	×	×	Fast attack detection	Complexity in selecting threshold
[34]	"Always ON" strategy	×	×	×	×	Reduced response time	Higher resource overhead
[35]	Firewall, load balancing	×	×	×	×	Reduced response time	Focus on specific DDoS type
[36]	Page separation MaxRW MinRW ERWS	×	~	×	×	Enhanced response time, reduced failed requests	Lack of discussion on the impact of undetected attacks
[37]	Bandwidth limits containerization	×	~	×	×	Improved availability	Requires additional machines Less effective against low-rate attacks
[38]	Nash equilibrium-based strategies	×	×	×	✓	Optimal malicious VM detection, limits impact on benign VMs.	Limited to standard cloud servers, serious delay
[39]	Markov chains, multi-valued decision diagrams	×	×	×	✓	Enhanced survivability analysis	Focused only on disk and system-level, miss variables affecting survivability
[40]	Attack-specific detectors with tolerance solutions	×	×	×	✓	Low-cost modelling framework	Not auto deployed, limited to design stage
[41]	Real-time detection	×	×	×	✓	Risk-based Monitoring	Limited to specific languages
[42]	Attack optimization	×	×	×	✓	New theoretical insights	lacks empirical validation, limited to specific queuing models
[43]	Self-healing, self-organization	✓	×	×	×	Assortativity metric	May miss all aspects of resilience
[44]	Cluster-based authentication	×	×	×	×	Improved accuracy	Time-consuming attack detection
[45]	Anomaly detection, filtering	×	×	×	×	Improved accuracy	computational complexity

(Continues)

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | (Continued)

Works	Tolerance techniques	Decentralized?	Adaptability?	Scalability?	Security analysis?	Advantages	Drawbacks
[46]	ML-based IDS	×	×	×	✓	Improved accuracy/response time	Limited to non-encrypted payloads, unable to resist adversarial ML attacks
[47]	SDN and binomial probability	×	×	×	×	Reduced EDoS attacks	No computational time analysis for controller
[48]	Rate limiting, traffic analysis	×	×	×	×	Cost-effective	Computational complexity, limited dataset
[49]	Decentralized data analysis	✓	×	×	×	Improved accuracy and classification	Potential bandwidth issues with false alarms
[50]	CNN-LSTM	×	×	×	×	Improved accuracy, and reduced false positives	Lack of realistic evaluation
[51]	Honey badger optimization, Bi-LSTM	×	×	×	×	High accuracy, advanced preprocessing	Detection speed, false positives
[52]	Recursive feature elimination, spatial clustering, time series analysis	×	×	×	×	Improved accuracy	Diverse real-world scenarios

✓: Yes ×: No ∼: Limited.

A DDoS attack is a malicious attempt to disrupt a server's normal functioning, making it unable to provide services to legitimate users. In this type of attack, the attacker controls multiple computers to launch a coordinated assault on the server. DDoS attacks are a significant threat to cloud resources as they generate large amounts of traffic, depleting critical computing resources like CPU, network bandwidth, and storage space. These attacks may also involve altering or destroying server configuration information and compromising network infrastructure equipment [55]. In fact, DDoS attacks have grown in terms of their scale and sophistication during 2018–2023. Concretely, such an attack against GitHub in 2018 reached a record of 1.35 Tbps, whereas in 2020, AWS of Amazon experienced an attack that touched as high as 2.3 Tbps. Finally, in 2023, this volume has reached 800 Gbps and 8.7 million requests per second, underscoring the increasing challenge for cybersecurity defences [56].

These attacks can cause service disruption, financial losses, reputational damage, operational costs, and legal implications for individuals and businesses if they fail to protect user data and ensure service availability. Cloud DDoS mitigation requires scalable and adaptable security solutions to address changing threat trends.

- **Blockchain and smart contract**

The appearance of Blockchain technology was a response to the centralized systems limitations, ensuring trust, transparency, and resilience. It was introduced in 2008 by Satoshi Nakamoto as the foundational infrastructure for Bitcoin, providing a decentralized framework for recording and verifying digital transactions without any need of intermediaries [57]. The great features of immutability, transparency, and distributed trust facilitated its swift integration across various sectors such as finance, energy, healthcare, smart city infrastructures, and cloud environments.

A blockchain is fundamentally a distributed wallet composed of a sequence of blocks, each block contains a list of validated transactions, a timestamp, a cryptographic hash of the previous block, a Merkle root for efficient transaction verification, a nonce (number used once), and other metadata [57, 58]. All participating devices have a replicated version of this chain; which allows eliminating any single point of failure and ensuring fault tolerance. Hence, altering historical data would require modifying all subsequent blocks across the majority of nodes, which is computationally impossible under standard consensus mechanisms. The blockchain systems consist of layers. The data layer organizes and secures transactions using hash chains and Merkle trees. The communications and transactions among peers are guaranty by the network layer. The consensus layer ensures agreement on a single valid version of the chain, even in the presence of malicious or failing nodes. The execution of SCs, enabling automated and self-enforcing operations, are all supported by the contract layer. Finally, the application layer offers end-user interfaces and services such as decentralized finance, identity verification, and distributed storage [58].

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The operational flow of a blockchain network is characterized by a series of well-defined steps that offer secure and decentralized transaction processing [57]:

- Transaction creation: a user initiates the transaction and signs it digitally with its private key ensuring authenticity and non-repudiation.
- Broadcasting: The signed transaction is sent through the peer-to-peer network to neighbouring nodes.
- Validation: The transaction is checked in each receiving node independently using predefined protocol methods, such as verifying the digital signature and checking account balances.
- Block formation: Each validated transaction is aggregated into a new block by a miner or validator node.
- Consensus execution: The network launches consensus algorithms to determine whether the proposed block is accepted.
- Ledger update: Once consensus is achieved, the new block is added to the existing blockchain.
- Replication: At the end. All nodes of the network receive the updated ledger, ensuring consistency and synchronization.

This decentralized, consensus-based workflow guaranty high levels of security, integrity, and auditability, even on unreliable risky environments.

The consensus mechanisms are the fundamental backbone of secure and decentralized operation of blockchain networks, they enable distributed nodes to agree on a single, valid ledger state [59]. These protocols determine how new blocks are appended and are critical for maintaining trust in decentralized systems. Over time, several mechanisms have emerged addressing the limitations of previous methods while ameliorating decentralization, efficiency, and scalability.

Nakamoto introduced PoW, the first consensus model designed to protect blockchain networks without a centralized authority. To ensure data integrity and impenetrability, miners should solve cryptographic puzzles. Nevertheless, PoW's high energy consumption, slowness, and increasing centralization due to mining pool dominance limit its applicability for large-scale or energy-sensitive applications [57].

To mitigate some of these problems, PoS selects validators based on how much cryptocurrency they are willing to lock as collateral. This approach shifts the necessary investment from computation to economic stake. It also improves scalability and reduces energy consumption. Still, PoS has issues of concentration of wealth, the "nothing-at-stake" problem, and other concerns over its sustainability in the long run [57].

For permissioned environments, PBFT was created to offer deterministic and quicker consensus. Through a voting procedure that accepts up to one-third of malevolent actors, it allows nodes to come to an agreement. Because of its communication overhead, PBFT is less practical in large networks despite having low latency and high throughput [58].

All things considered, the development of consensus mechanisms shows an ongoing attempt to strike a balance between performance, decentralization, and trust presumptions. The context of the system, including its governance structure, threat model, and scalability requirements, greatly influences the protocol selection [57, 58].

In a blockchain system, blocks record transaction information to ensure reliability and safety. SCs are self-verifying, self-executing, and tamper-resistant, serving as voting mechanisms for arbitration committee decisions, industrial standards approval, and member addition.

SCs, introduced by Nick Szabo, enforce contract terms and reduce reliance on centralized trust agencies. SCs benefit from decentralization, openness, transparency, and tamper-resistance, allowing participants to trade and collaborate without trust [60, 61].

Integrating blockchain and SCs into cloud computing can improve interactions and transactions, enhancing security and efficiency while mitigating vulnerabilities associated with centralized attacks and misinformation.

Based on their control structures and authentication models, blockchain systems can be broadly divided into three categories: consortium, private, and public [59]. Public blockchains are accessible to all users and are completely decentralized. Transactions can be read, written, and validated by any node; PoW is usually used for consensus. Public blockchains are vulnerable to 51% attacks, have scalability problems, and consume a lot of energy, despite their ability to guaranty transparency and tamper resistance [58].

On the other hand, private blockchains limit access to a specific set of users. Trusted entities oversee transaction validation, which frequently uses practical Byzantine fault tolerance (PBFT) for effective consensus. This model, which is typically used in institutional or enterprise settings, decreases decentralization even though it provides better performance and privacy [58].

Consortium blockchains operate as a hybrid model and are managed by a collection of organizations. Consensus decisions are typically shared using multi-signature mechanisms. This structure improves scalability and operational efficiency, but it also increases the risk of validator collusion, which could compromise immutability [58].

This classification allows blockchain systems to be deployed in a wide range of scenarios, from fully decentralized public infrastructures to tightly controlled enterprise ecosystems, supporting a wide range of distributed and cloud-native applications.

In large-scale cloud computing environments, blockchain enhances attack tolerance and resilience, especially against DDoS threats. Smart contracts automate resource governance, access control, and adaptive threat responses, and its decentralized architecture eliminates the need for central control points. When combined with hierarchical risk models, blockchain provides dynamic, scalable security features suitable for modern distributed architectures [58].

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

- Dempster–Shafer theory

DST, introduced by Arthur P. Dempster and Glenn Shafer in the 1970s, is a mathematical theory of evidence that allows for the representation and combination of uncertain and imprecise information [62]. It is typically used in situations where there is a degree of uncertainty, integrating information fed from multiple sources, and making decisions using information of an unknown nature. This theory was established for a specific number of items in the universal sets. It is based on the concept of belief functions, which quantify the degree of belief in a given proposition or set of propositions. In DST, key concepts crucial for managing uncertainty include:

- **Belief function** (Bel)

This function assigns values from 0 to 1 to each subset of a frame of discernment, quantifying the degree of belief that the true outcome lies within that subset based on the available evidence. The belief assigned to a subset A is calculated by summing the masses assigned to all focal elements that are subsets of A. For a subset

A, the belief is calculated as:

$$\mathrm{Bel}(A) = \sum_{B \subseteq A} m(B) \tag{1}$$

Where $m(B)$ represents the basic probability assignment given to focal elements B that are subsets of A.

- **Plausibility function** (Pl)

This function complements the belief function by assigning values to subsets, reflecting the belief that the true outcome is not in their complements. It ranges from 0 to 1, with higher values indicating a greater degree of potential for the outcome to be within the subset. For a subset A, the plausibility is calculated as:

$$Pl(A) = 1 - \mathrm{Bel}(A^c) \tag{2}$$

Where A^c represents the probability that the true outcome is not outside of A.

- **Focal element**: represents subsets with non-zero belief, denoting propositions considered most plausible based on evidence. These focal elements allow for the computation of belief and plausibility functions and concentrate the uncertainty study on subsets that are more basic in the frame of discernment.

These concepts form the foundation of DST, enabling structured reasoning in uncertain and ambiguous decision-making scenarios [63, 64].

We chose DST for this work, instead of alternative methods like Bayesian inference or fuzzy logic because of its specific advantages in dealing with the complexities of multi-level DDoS risk assessment in dynamic cloud environments. Firstly, DST's ability to explicitly represent epistemic uncertainty is crucial when node data is sparse, incomplete, or missing, avoiding the need for potentially inaccurate prior probability assignments required by Bayesian methods. Secondly, Dempster's rule of combination provides a flexible and robust mechanism for aggregating evidence from diverse, potentially conflicting sources (e.g. CPU load, network patterns, node reputation) across the local, cluster, and global layers, aligning well with our hierarchical, multi-indicator risk assessment. While fuzzy logic excels at handling linguistic imprecision, DST provides a more structured framework to quantify belief and plausibility based on evidence subsets, which is more suitable for combining the varied risk indicators within our system. Preliminary modelling confirmed DST's effectiveness in handling the inherent uncertainty and heterogeneity of risk data in this context.

In our proposition, we apply the DST to SCs for dynamic risk assessment and threshold adjustment. These self-executing contracts use DST concepts to assess and respond to risks in real-time. By aggregating data from cloud nodes, the contracts calculate belief values using the belief function, estimate potential risks using the plausibility function, and use focal elements and the mass function for accurate risk assessment. This integration enhances the system's responsiveness and resilience against cloud attacks, ensuring effective and reliable risk management.

4 | Proposition

The main objective of our model is presented in this section. We propose a novel blockchain framework that merges SCs with a hierarchical risk-based attack tolerance mechanism to improve the cloud security via dynamic DST risk assessment and adaptive thresholds. The proposed model aims to provide a comprehensive and adaptive defence mechanism against various types of attacks in a cloud environment. The model incorporates a formal approach, assessing both user activities and server states to generate a detailed risk profile. This risk profile dynamically adapts to ongoing conditions within the cloud environment and is assessed in a distributed, parallel manner across clusters of nodes, as well as globally across the entire cloud environment.

4.1 | System Architecture

This section presents the hierarchical, blockchain-integrated architecture highlighting our novel framework for cloud attack tolerance. Figure 1 provides an outline of our approach:

The proposed system architecture is represented within a graph $G = (V, E)$ modelling a cloud computing environment, where V is the set of vertices (nodes), and $E \subseteq V \times V$ is the set of edges (communication links).

Each node $v \in V$ is characterized by a tuple $(\mathrm{id}_v, T_v, \mathrm{A}_v, \mathcal{R}(\mathrm{A}_v))$, where:

- id_v: is the unique identifier of the node.
- T_v: Type of node v, which can be one of the following: user, server, miner, or cluster head. For nodes with multiple roles (e.g. a cluster head that is also a miner), T_v will include all applicable types. For example, T_v= {cluster head, miner}

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

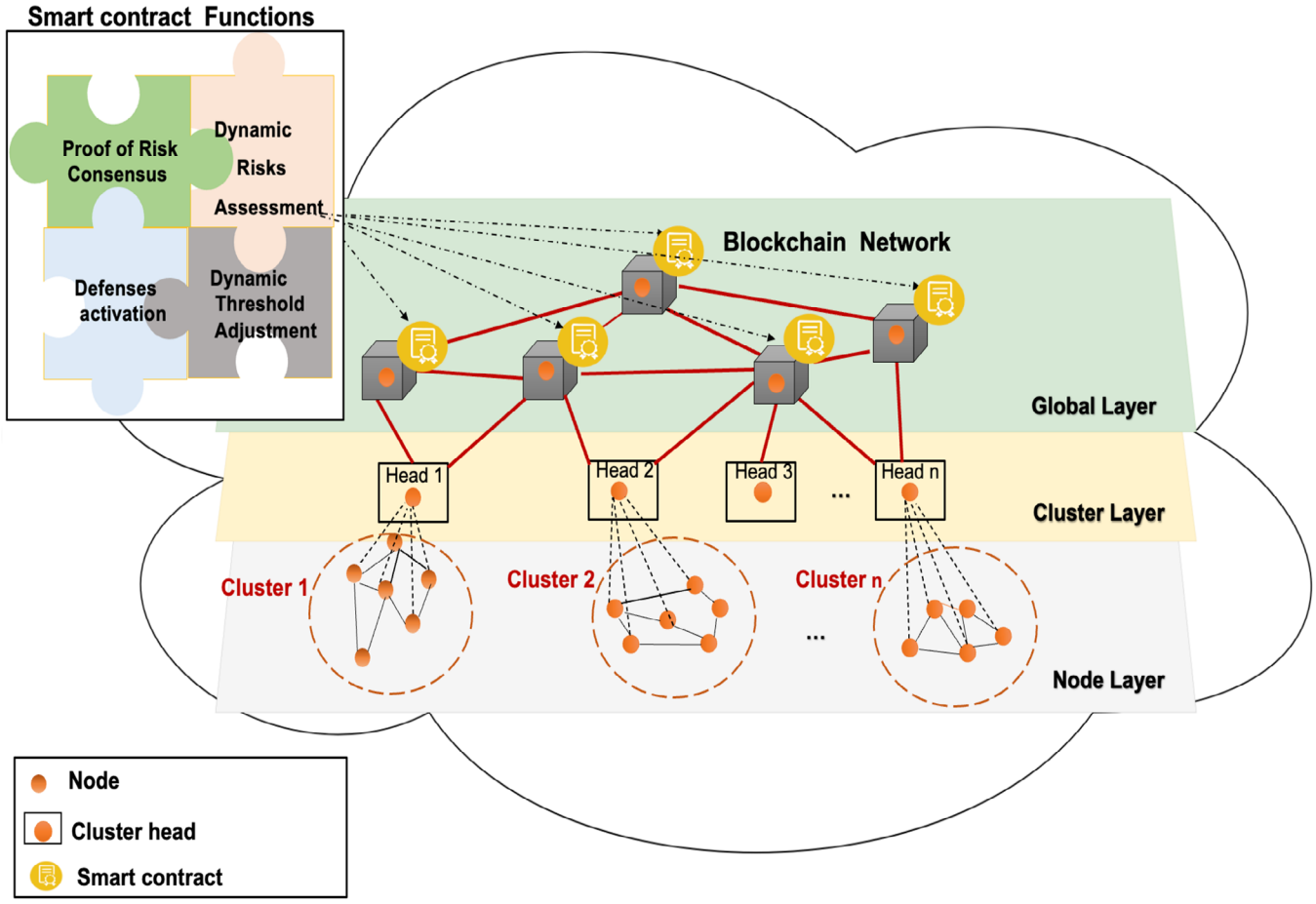


FIGURE 1 | Proposed system architecture.

- $A_v = \{a_{state}, GS_{deg}, GS_{cent}, CQ_{bw}, NH_{inc}, TP_{freq}, NT_a, NR_u, NR_{ud}, AU_{NR}, V_S\}$ is the set of attributes for node v, incorporating:
  - a_{state} The operational state of the node (active, idle, under-attack etc.), similarly encoded.
  - GS_{deg}: Graph Structure attributes considering the node's degree, centrality, and other topological characteristics.
  - GS_{cent}: A centrality measure (could be betweenness, closeness, or eigenvector centrality) that captures the node's importance in the cloud network's flow.
  - CQ_{bw}: Bandwidth of the node's connections, representing the quality and capacity of its communication links.
  - NH_{inc}: A count of security incidents involving the node, reflecting its historical vulnerability.
  - TP_{freq}: Frequency of transactions, indicating the node's activity level and potential exposure to risk.
  - NT_a: Node traffic anomalies, monitoring for sudden spikes or unusual patterns in cloud traffic, serving as a direct indicator of potential DDoS attacks.
  - NR_u: Node resource usage, observing high or unusual usage of resources such as CPU and memory, which might indicate a node is struggling under a DDoS attack.
  - NR_{ud}: Analyse the distribution of resource usage across different tasks and processes.
  - AU_{NR}: Monitor the average resource usage of the node.
  - V_S: Evaluate the node's ability to dynamically scale resources.

Each attribute is a ratio, providing a normalized measure of the specific risk aspect associated with node v. Integrating a broad spectrum of attributes such as NR_u, NR_{UD}, AU_{NR}, and V_S, some of which are inspired by insights from [65], enhances the model's efficiency and expands its capacity to handle diverse attack tolerances, particularly in resource utilization and scalability. This strategic enhancement equips the system to adeptly identify, manage, and mitigate risks, fostering a cloud computing environment that is robust, resilient, and highly fault-tolerant against various attack scenarios.

- The risk assessment function is defined as:

$$\mathcal{R} : A_v \rightarrow \mathbb{R} \tag{3}$$

Where $\mathcal{R}(A_v)$ assigns a risk value r_v to node v based on its attributes.

- The formalized defence mechanisms are incorporated as follows:

1) **Local layer defence (layer 1)**

- $L : V \rightarrow \mathcal{F}$, where $L(v)$ is activated if $\mathcal{R}(A_v) > \theta_L(v)$, triggering local defense mechanisms.
- $$L(v) = \begin{cases} \{RL(v,p), TF(v,f), RA(v,r)\} & \text{if } \mathcal{R}(A_v) > \theta_L(v), \\ \varnothing & \text{otherwise} \end{cases} \tag{4}$$

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$L(v)$ is a composite function that activates specific defence mechanisms based on the risk assessment $\mathcal{R}(\mathrm{A}_v)$ of each node v. If the risk surpasses the threshold $\theta_{\mathrm{L}}(v)$, the function activates the appropriate defense mechanisms. RL(v,p) rate-limiting for node v with parameter p. This limits traffic coming to and from the node, which is crucial for mitigating DDoS attacks. The parameter p specifies details of the process of rate limiting. TF(v, f) represents the traffic filtering function. f denotes the filtering criteria. This parameter defines how the traffic is analysed and filtered, determining which traffic is considered malicious and should be blocked to protect the node. RA(v, r) Resource Allocation function for node v, r. represents the resource allocation parameters. This parameter controls the reallocation and adjustment of the node's resources, including bandwidth and computing power, particularly i when the node is under attack. The aim is to ensure that the node can maintain its functionality and service availability even when facing resource-intensive threats.

Every node within the cloud computing environment is endowed with a customized defence strategy, meticulously aligned with its unique risk profile. This strategic alignment significantly bolsters the system's overall security and fortifies its resilience against potential threats.

2) **Cluster layer defence (layer 2)**

- $C = \{C_1, C_2, C_3, \ldots, C_k\}$, a partition of V into clusters with $C_i \subseteq \mathrm{V}$ and a cluster head $h_i \in C_i$.
- CR : $\wp(V) \to \mathbb{R}$, where:

$$\mathrm{CR}C_i sum_{v \in C_i} \mathcal{R}(\mathrm{A}_v) \tag{5}$$

- CD : $\wp(V) \to \mathcal{F}$, activated if $\mathrm{CR}(C_i) > \theta_{\mathrm{C}}(C_i)$, triggering federated cluster mechanisms.

  - $\mathrm{CD}(C_i) =$

$$\begin{cases} \mathrm{CAC}(C_i, s_{\mathrm{status}}, \mathrm{TR}(C_i, t), \mathrm{NI}(C_i, N)\mathrm{DPA}(C_i, P) & \text{if } \mathrm{CR}(C_i) > \theta_{\mathrm{C}}(C_i) \\ \emptyset & \text{otherwise} \end{cases} \tag{6}$$

  $\mathrm{CAC}(C_i, s_{\mathrm{status}})$: Cluster-wide alert and coordination activated when a threat is detected.

$\mathrm{TR}(C_i, t)$: Traffic rerouting, adjusts traffic flow within the cluster.

$\mathrm{NI}(C_i, N)$: Node isolation (isolates affected or vulnerable nodes).

$\mathrm{DPA}(C_i, P)$: Defense protocol activation, activates relevant defence mechanisms across the cluster.

$\mathrm{CD}(C_i)$ is a composite function that activates specific cluster-wide defense mechanisms based on the collective risk assessment $\mathrm{CR}(C_i)$ of the cluster. If the risk exceeds the threshold $\theta_{\mathrm{C}}(C_i)$, the function activates a coordinated response involving alert systems, traffic management, node isolation, and the activation of predefined defence protocols. This approach ensures a robust and adaptive defence mechanism at the cluster level, enhancing the overall security and resilience of the cloud environment against a range of threats.

3) **Global layer defence (layer 3)**:

- GR : $\wp(V) \to \mathbb{R}$, where:

$$\mathrm{GR}(G) = \sum_{C_i \in C} \mathrm{CR}(C_i) \tag{7}$$

- GD : $\wp(V) \to \mathcal{F}$, activated if $\mathrm{GR}(G) > \theta_{\mathrm{G}}$, triggering federated cluster mechanisms.

- **Smart contract functions**:

- **Dynamic risks assessment**:

$\mathrm{SC}_{\mathrm{RA}}$: $P(C) \times \mathcal{G} \to \mathbb{R} \times \mathbb{R}$. This smart contract function assesses risk at both the cluster and global levels, with $P(C)$ representing all possible cluster combinations and $\mathcal{G}$ representing the global graph states. The smart contract records and updates the risk scores on the blockchain, ensuring a transparent and tamper-proof risk assessment.

- **Dynamic threshold adjustment**:

$\mathrm{SC}_{\mathrm{DT}}$: $\mathbb{R} \times \theta \to \theta'$. This function adjusts the system threshold based on the current global risk score $\mathbb{R}$. It takes two inputs (θ) as: the threshold and the aggregated risk level of the cloud, then, it optimizes the threshold set by adjusting thresholds θ'. This process ensures system responsiveness to risk level changes and adaptive capabilities against emerging threats or system operation aspects as well.

- **Consensus nodes selection function**:

$\mathbf{SC_{CNS}}$: $\mathbb{R} \times V \to V_{\mathrm{selected}}$. The function assigns priority to nodes with reduced risk scores when selecting them for consensus. V represents the set of blockchain nodes, while V_{selected} denotes the set of nodes that are selected for consensus.

- **Proof of risk consensus function**

$\mathbf{SC_{PoR}}$: $V_{\mathrm{selected}} \times \mathcal{T} \to \mathcal{C}$. This function implements a consensus mechanism that considers the risk profile of nodes. It uses transactions $\mathcal{T}$ and the consensus outcome $\mathcal{C}$ is the consensus outcome. Nodes in V_{selected} validate transactions, with their voting power inversely proportional to their risk scores. The smart contract governs the proof of risk (PoR) mechanism, ensuring a fair and secure validation process based on the risk-adjusted influence of participating nodes.

- **Attacks detection and defences**:

$\mathbf{SC_D}$: $A_v \times \tau \to F$. This function analyses node activities and triggers defence mechanisms when attacks are detected. It uses attributes A_v of each node and transaction history τ to identify unusual patterns, improving attack detection. The set F represents the defence mechanisms that will be activated.

- The hierarchical defence mechanism is represented by a decision function, denoted as $\mathcal{D}$. This function determines the level of response based on the risk assessment.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Basic versus enhanced blockchain.

Feature		Basic blockchain	Enhanced blockchain
Structural aspect	Data storage	Linear block storage with transaction data.	Advanced storage with integrated risk assessment data and dynamic security updates.
	Network topology	Typically, a peer-to-peer network.	Advanced network design incorporating node clusters with differentiated roles based on risk assessment.
	Block structure	Standard block structure containing transaction data and a cryptographic hash.	Enhanced block structure, including detailed risk scores and security protocol states, enabling dynamic response to threats.
	Inter-block communication	Limited to the verification of transactions.	Extended to include communication protocols for risk data sharing and consensus decision-making based on security metrics.
	Scalability	Often challenged by the size and complexity of the network.	Structurally optimized for scalability, facilitating extensive parallel processing for risk assessment and defence activation.
	Integration with cloud infrastructure	Not specifically designed for cloud integration.	Customized integration with the cloud, ensuring smooth interaction and enhanced security.
Functional aspect	Smart contracts	Automated execution of agreements.	Advanced algorithms for risk assessment and dynamic security adjustments.
	Consensus mechanism	PoW or PoS.	Proof of risk prioritizing nodes with lower risk scores for enhanced security.
	Risk assessment	Limited or absent.	Integrated multi-layered approach (node, cluster, global) using DST for comprehensive risk evaluation.
	Threshold adjustment	Static security parameters.	Dynamic adjustment using Dempster–Shafer theory, responsive to changing risk levels.
	Defence activation	Typically, predefined conditions.	Decentralized, parallel, and responsive to assessed risk levels, enhancing cloud attack tolerance.
	Decentralization	Fundamental characteristic.	Enhanced through decentralized risk assessment, defences activate and dynamic consensus approach.

$$\mathcal{D}(v,\mathcal{R})\begin{cases} L(v), & if\ R(\mathrm{A}_v) > \theta_{\mathrm{L}}(v) \\ CD(C_i), & if\ \exists\, C_i \in C\ :(v \in C_i)\Lambda(CR(C_i)\ \theta_{\mathrm{C}} \\ GD(G), & if\ GR(G) > \theta_{\mathrm{G}} \end{cases} \quad (8)$$

4.2 | Enhanced Blockchain for Improved Cloud Attack Tolerance

Our novel blockchain framework is introduced in this section. Table 2 highlights the structural and functional advancements of our enhanced blockchain with traditional design.

According to comparative analysis in Table 2, data storage, risk assessment, and security updates have been with enhanced blockchain, which make it more resilient to DDoS attacks from the cloud. The enhanced blockchain also uses a network topology with node clusters based on risk assessment to enhance defensive capabilities. The Proof of Risk consensus mechanism helps identify lower-risk nodes, resulting in a more secure network. Advanced SCs enable real-time risk assessment and security adjustments. These improvements enhance defence against cloud-based attacks while maintaining decentralization, transparency, and immutability.

4.2.1 | Smart Contract Functions

This sub-section describes the five smart contract cores (SCs) that form the backbone of our blockchain. The proposed system employs SCs and algorithms to estimate risk and adjust thresholds at each node, cluster, and global levels, as detailed in Figure 2. This process precedes the selection of consensus nodes and the implementation of a Proof of Risk consensus mechanism.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

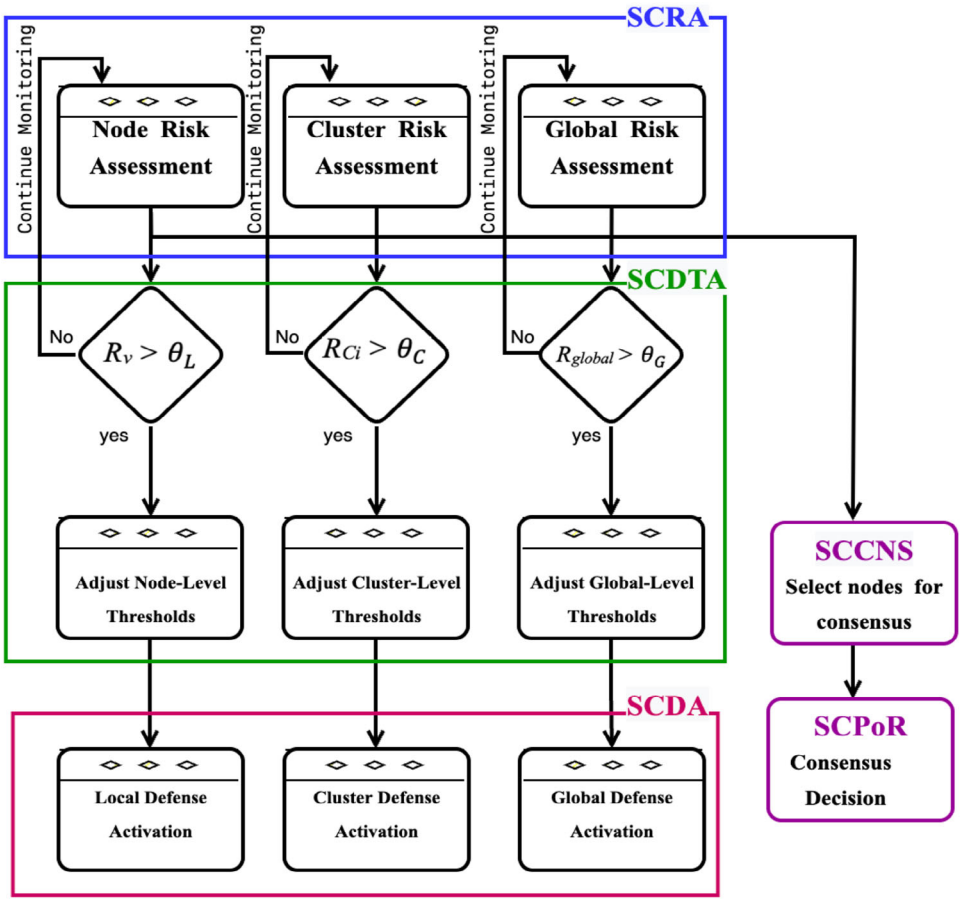


FIGURE 2 | Workflow of smart contract functions.

- **Smart contract risk assessment** (SCRA)

Algorithm 1 is a multi-layered approach used to assess risks in the cloud. It uses the DST to evaluate risks in a decentralized and efficient way. The method checks the risk through the three levels: node-level, cluster-level, and global-level.

The algorithm begins with a node-level risk assessment using basic probability assignments (BPAs) to assign belief values to the attributes A_v of each node. These beliefs are combined using Dempster's rule to calculate a comprehensive risk score R_v. Then, the scores are further normalized to $R_{v\text{-norm}}$ and stored on the blockchain, ensuring immutability and transparency of the recorded risk levels for each node.

Next, the algorithm launches parallel cluster-level risk assessment for the cluster. It retrieves the normalized node risk scores ($R_{v\text{-norm}}$) from the blockchain, and uses them to assign belief values based on Dempster's rule. This produces a comprehensive and dynamic cluster risk score (R_{Ci}), which is then normalized and stored on the blockchain. Furthermore, and by combining the normalized cluster risk scores ($R_{Ci\text{-norm}}$) using Dempster's rule. the algorithm performs global risk assessment and generates a global risk score (R_{global}) that represents the combined and interactive assessment of all clusters. This score is normalized and stored on the blockchain, providing a more detailed and comprehensive global risk assessment. The algorithm dynamically adjusts security protocols based on these detailed risk assessments and records all data on the blockchain. The use of DST in dynamic and interactive risk assessment further strengthens the security management system.

- **Smart contract dynamic threshold adjustment (SCDTA)**

In a smart contract, Algorithm 2 adjusts thresholds based on risk scores and scaling factors. The system employs dynamic thresholds based on belief and plausibility values. These thresholds are continuously optimized through adaptive scaling factors (ξ, ζ, γ, δ, α, β) controlled by smart contracts, which adjust sensitivity based on real-time contextual data (node, cluster, global states), historical trends, and feedback from defence results, thus, ensures responsiveness in dynamic environments.

The algorithm first adjusts node-level thresholds (θ_L) by calculating belief and plausibility scores for each node and applying scaling factors ξ and ζ. Then, it adjusts cluster-level thresholds (θ_C) by calculating belief and plausibility scores for each cluster and applying scaling factors γ and δ. Finally, it adjusts global-level thresholds (θ_G) by calculating belief and plausibility scores for the entire system and applying scaling factors α and β. The blockchain aggregates and stores the adjusted thresholds. When the risk level is considered high, a defence activation function is activated.

Figure 3 shows how adjusted thresholds in the cloud can effectively enhance attack tolerance. This approach involves dynamically adjusting security measures based on risk profiles to

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 | Smart contract risk assessment with parallel processing.

Input: $G = (V, E)$, A_v for each v in V, Cluster set C, Threshold sets θ_L, θ_C, θ_G

Output: Normalized risk scores for nodes, clusters, and global environment

Begin

Node-level risk assessment

Parallel for each v in V **do**

Initialize m_v: $2^{A_v} \rightarrow [0,1]$

For each $a \in A_v$ **do**

$m_v(a) = \text{AssignBeliefValue}(a)$

$m_{v\text{-combined}} = \text{DempsterCombination}(m_v)$

$R_v = \text{CalculateRiskScore}(m_{v\text{-combined}})$

$R_{v\text{-norm}} = \text{NormalizeRiskScore}(R_v, V)$

StoreRiskScore($R_{v\text{-norm}}$, v, Blockchain)

End

Cluster-level risk assessment

Parallel for each $(C_i \in C)$ **do**

Initialize m_{Ci}: $2^{ACi} \rightarrow [0,1]$

For each v in C_i **do**

$R_{v\text{-norm}} = \text{RetrieveNormalizedNodeRisk}(v, \text{Blockchain})$

$m_{Ci(v)} = \text{AssignNodeBeliefValue}(R_{v\text{-norm}}, v, C_i)$

$m_{Ci\text{-combined}} = \text{DempsterCombination}(m_{Ci})$

$R_{Ci} = \text{CalculateClusterRiskScore}(m_{Ci\text{-combined}})$

$R_{Ci\text{-norm}} = \text{NormalizeRiskScore}(R_{Ci}, C)$

StoreClusterRiskScore($R_{Ci\text{-norm}}$, C_i, Blockchain)

End

Global risk assessment

Initialize m_{Global}: $2^{A_\text{Global}} \rightarrow [0,1]$

Parallel for each Ci in C **do**

$R_{Ci\text{-norm}} = \text{RetrieveNormalizedClusterRisk}(C_i, \text{Blockchain})$

$m_{\text{Global}}(C_i) = \text{AssignClusterBeliefValue}(R_{Ci\text{-norm}}, C_i)$

$m_{\text{Global-combined}} = \text{DempsterCombination}(m_{\text{Global}})$

$R_{\text{global}} = \text{CalculateGlobalRiskScore}(m_{\text{Global-combined}})$

$R_{\text{global_norm}} = \text{NormalizeGlobalRiskScore}(R_{\text{global}})$

StoreGlobalRiskScore($R_{\text{global_norm}}$, Blockchain)

End

Threshold adjustment decision

Parallel for each (v in V) **do**

If ($R_{v\text{-norm}} >$ Threshold (θ_L, v)) **then**

AdjustNodeSecurityProtocol(v)

Parallel for each ($C_i \in C$) **do**

If ($R_{Ci} >$ Threshold, (θ_C, C_i)) **then**

AdjustClusterSecurityProtocol(C_i)

If ($R_{\text{global_norm}} >$ Threshold (θ_G))**then**

AdjustGlobalSecurityProtocol()

End

Return $\{R_{v\text{-norm}} \mid v \in V\}$, $\{R_{Ci} \mid C_i \in C\}$, $R_{\text{global_norm}}$

End

ALGORITHM 2 | Smart contract dynamic threshold adjustment function.

Input: R_{global}, R_{cluster}, R_{local}, θ_L, θ_C, θ_G

Output: Adjusted thresholds θ'_L, θ'_C, θ'_G

Begin

Adjust node-level thresholds

For each (θ_{Li} in θ_L) **do**

$\text{id}_v = \text{GetNode}_{\text{Id}}(\theta_{Li})$

If (NodeExists(id_v, blockchain) = false) **then**

Return Error("Node does not exist")

$R_{\text{local}} = \text{GetLocalRiskScore}(\text{id}_v)$

$\text{Bel}_v(R_{\text{local}}) = \text{CalculateBelief}(R_{\text{local}}, A_v)$

$\text{Pl}_v(R_{\text{local}}) = \text{CalculatePlausibility}(R_{\text{local}}, A_v)$

$\xi, \zeta = \text{GetScalingFactorsNode}()$

$\theta'_L = \theta_L \times (\xi * \text{Bel}_v(R_{\text{local}}) + \zeta * \text{Pl}_v(R_{\text{local}}))$

End

Adjust cluster-level thresholds

For each (θ_{Ci} in θ_C) **do**

$\text{id}_c = \text{GetClusterId}(\theta_{Ci})$

If (ClusterExists(id_c, blockchain) = false) **then**

Return Error("Cluster does not exist")

$R_{\text{cluster}} = \text{GetClusterRiskScore}(\text{id}_c)$

$\text{Bel}_{Ci}(R_{\text{cluster}}) = \text{CalculateBelief}(R_{\text{cluster}}, C(C_i))$

$\text{Pl}_{Ci}(R_{\text{cluster}}) = \text{CalculatePlausibility}(R_{\text{cluster}}, C(C_i))$

$\gamma, \delta = \text{GetScalingFactorsCluster}()$

$\theta'_{Ci} = \theta_{Ci} \times (\gamma * \text{Bel}_{Ci}(R_{\text{cluster}}) + \delta * \text{Pl}_{Ci}(R_{\text{cluster}}))$

End

Adjust global-level thresholds

$\text{Bel}(R_{\text{global}}) = \text{CalculateGlobalBelief}(R_{\text{global}})$

$\text{Pl}(R_{\text{global}}) = \text{CalculateGlobalPlausibility}(R_{\text{global}})$

$\alpha, \beta = \text{GetScalingFactorsGlobal}()$

For each (θ_{Gi} *in* θ_G) **do**

$\theta'_{Gi} = \theta_{Gi} \times (\alpha * \text{Bel}(R_{\text{global}}) + \beta * \text{Pl}(R_{\text{global}}))$

$\theta' = \text{AggregateThresholds}(\theta'_L, \theta'_C, \theta'_G)$

StoreThresholds(θ', blockchain)

If (CheckRiskLevel(θ')) then

Activate defence activation function

Return θ'

End

(Continues)

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

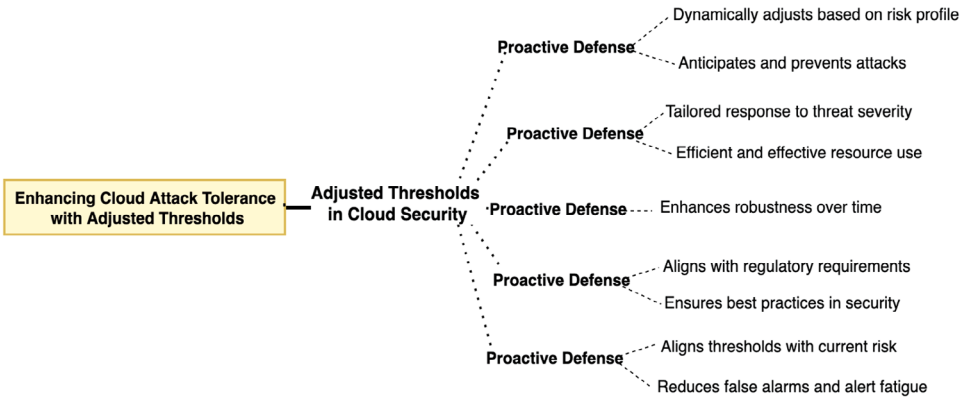


FIGURE 3 | Enhancing cloud attack tolerance with adjusted thresholds.

enhance defence against potential attacks and optimize resource utilization. By proactively adapting responses to specific threats, the system becomes more robust and responsive. It also aligns with regulatory standards and best practices to maintain trust and compliance. Adjusting risk thresholds also helps reduce false alarms.

Threshold adjustments are crucial for dealing with current and future threats in a proactive and adaptive manner. This approach enhances the defence of cloud environments against evolving threats and improves their ability to withstand and recover from attacks. By continuously aligning security measures with current risk profiles, cloud environments can effectively prepare for potential threats.

- **Smart contract defences activation function (SCDA)**

Algorithm 3 employs many defence mechanisms at multiple levels to guarantee complete and adaptable security. In the local layer, when nodes exceed the risk threshold θ_L, specific defenses based on their unique attributes A_v are activated. These include rate-limiting, traffic filtering, and IP address filtering, ensuring effective responses while maintaining efficiency. This system dynamically applies defences, enhancing cloud resilience and security. In the cluster layer, if the risks of node clusters exceed threshold θ_C, cluster-wide defenses are activated. These include coordination mechanisms (CAC), such as alert dissemination and response coordination, to address threats in a unified manner. Additionally, traffic rerouting (TR) and node isolation (NI) techniques are used to manage traffic flow and protect vulnerable nodes. At the global level, exceeding the risk threshold θ_G initiates comprehensive defense actions (GA). These include TR and NI to manage data flow and protect the integrity of high-risk or compromised clusters on a global scale.

All defence actions across levels are securely recorded on the blockchain, ensuring transparency and enhancing cloud security against a variety of threats.

- **Smart contract algorithm: consensus nodes selection function (SCCNS)**

The SCCNS algorithm is vital to our system. The selection of nodes for consensus is based on their risk scores. This prioritizes nodes with lower scores, ensuring the selection of secure and stable nodes for transaction validation. This approach enhances the integrity and reliability of the consensus mechanism.

- **Smart contract proof of risk consensus function (SCPoR)**

The SCPoR is a new consensus algorithm designed for achieving consensus among selected nodes in cloud computing environments. This algorithm involves a risk assessment and influence weighting step, where each node's risk score is calculated and used to determine its influence weight. The SCPoR algorithm ensures stable and secure consensus by dynamically adjusting to nodes' security profiles, ensuring fair and secure participation by assessing nodes on security, performance, and vulnerability. Additionally, SCPoR ensures impartiality in the consensus process, enabling every node to participate honestly according to their security criteria. SCPoR not only enhances cloud security but also enables a more comprehensive blockchain ecosystem. It enhances cloud security and stability by giving priority to nodes with lower risk scores. This will enhance resilience against a range of attacks and will always ensure efficiency by choosing stable nodes, even in the presence of security threats.

5 | Simulations and Performance Analysis

In the next section, we evaluate how well our proposed system can resist attacks while enhancing reliability and availability in a massive 10,000-node cloud. We simulate several high-impact DDoS attacks, including ICMP, Slowloris, Smurf, HTTP Flood, SYN Flood, UDP Flood, and Malformed Packet scenarios, to evaluate system performance under intense traffic-based threats. To demonstrate broader applicability beyond DDoS, we also include Sybil attacks, insider threats, data poisoning, and advanced persistent threats (APTs), including zero-day attacks. We show the effectiveness of our system in comparison to other systems that are currently in use, including IDARP [11], SDMTA [14], HIDM [16], OCB [17], SaE-ELM [19], DMD-MTD [20], LRDADF [21], MAD-MLT [22], BaysCNN [26], AT-CWS [41] and FACVO-DNFN [45], OpenStackDP [46], BFL-CIDS [49], DeepDefend [50], and RDAER [52].

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 3 | Smart contract defences activation function.

Input: $G = (V, E)$, R_v for each node $v \in V$, R_{Ci} for each cluster C_i, and R_{global} for the global environment. θ_L, θ_C, θ_G

Output: Activate a set of defence actions for each layer

Begin

Parallel local layer defence activation (layer 1):

Parallel for each $(v \in V)$ **do**

If $R_v > \theta_L$ **then**

// Activate defences based on node attributes A_v

$L(v) = \{RL(v, p)$, // rate-limiting scaled based on transaction frequency (TP_{freq})

//and traffic anomalies (NT_a).

$TF(v, f)$, // Blocks malicious patterns using historical incidents (NH_{inc})

//and traffic anomaly signatures ((NT_a).

IPF$(v, ip$ // Ban IPs with frequent NT_a spikes

$)\}$

Else

$L(v) = \varnothing$

End

Parallel cluster layer defence activation (layer 2):

Parallel for each (C_i) **do**

If $R_{Ci} > \theta_C$ **then**

// Activate defences based on cluster attributes

$CD(C_i) = \{CAC(C_i, s)$, // Alert nodes with high centrality (GS_{cent})

$TR(C_i, t)$, // Redirects traffic to paths with high bandwidth quality (CQ_{bw}).

$NI(C_i, n)$. // Isolate nodes with abnormal historical incidents (NH_{inc})

}

else

$CD(C_i) = \varnothing$

End

Global layer defence activation (layer 3):

If $R_{global} > \theta_G$ **then**

// Activate defences based on global attributes A_Global

$GD(G) = \{GA(G, a)$, //Broadcasts alerts for network-wide traffic anomalies.

$TR(G, t)$, // Reroute traffic from low-GS_{deg} regions

$NI(G, n)$. // Blacklists nodes with critical resource usage

}

else

$GD(G) = \varnothing$

End

Store defence activation (blockchain)

End

5.1 | Simulation Setup

Our implementations and experiments were conducted on an Apple M1 chip computer with 8 GB of RAM. We used Python Jupyter Notebook as our primary development environment, integrating various libraries and tools for system construction and evaluation.

We implemented Hyperledger Fabric for blockchain infrastructure using the fabric SDK for Python (hfc-python). This setup involved configuring fabric peers and ordering nodes, defining smart contracts, and managing ledger changes through Python scripts, with Docker managing fabric components in a local development environment for effective deployment and transaction execution.

For cloud structure and scalability, we designed a large-scale cloud simulation using NetworkX to model hierarchical cloud topologies and manage complex network, and SimPy for discrete-event simulation. Python's socket library and ZeroMQ were employed to enable efficient communication between nodes. SciPy and SymPy supported advanced numerical for implementing DST. Multiprocessing and concurrent futures were employed for parallel processing to enhance computational efficiency. We tested scalability by varying network size from 1000 to 10,000 nodes including users, servers, and cluster heads to reflect realistic cloud scenarios. We also varied malicious node proportions from 0% to 80% to evaluate system performance under increasing attack levels. At each scale and stress level, key metrics were tracked to assess system adaptability and robustness.

In order to mimic real cloud behaviour, the attack simulations were conducted using Python's socket and Scapy libraries to generate synthetic traffic, including legitimate activities and various cyberattacks such as ICMP Flood, UDP Flood, SYN Flood, HTTP Flood, Slowloris, Smurf Attack, malformed packets, Sybil attacks, insider threats, data poisoning, replay attacks, and advanced persistent threats. The structure and timing of these attacks were guided by patterns observed in the CIC-IDS2017 and CERT Insider Threat datasets, which includes a wide spectrum of relevant threats. Each simulation logged time-stamped network traffic, packet metadata, and behavioural events, producing a dataset capturing node states, traffic attributes, resource metrics, and blockchain behaviour. These logs, which enable binary and multiclass classification, allow for a thorough assessment of system resilience in a large-scale cloud environment.

Pandas and NumPy were used for data manipulation and analysis, and Matplotlib and Seaborn provided visualization tools for result interpretation.

Besides those tools in Python, we also utilized some external tools for blockchain development and security. We utilized Remix IDE, a Web-based Ethereum development environment, to develop and test solidity-based smart contracts, validate blockchain interactions, and ensure contract performance. A Haskell-based smart contract testing tool, Echidna, was employed to identify vulnerabilities and assess security through generating test cases that ensured the robustness of our contracts. The integration of various tools such as Hyperledger Fabric, Docker, and external

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 | Summary of evaluation metrics.

Metric	Brief description	formula
False positive rate (FPR)	Percentage of wrong detected attacks.	$\text{FPR} = \frac{\text{FP}}{\text{TN} + \text{FP}}$ (9)
False negative rate (FNR)	Percentage of undetected attacks.	$\text{FNR} = \frac{\text{FN}}{\text{TP} + \text{FN}}$ (10)
Attack detection rate (ADR)	Percentage of correctly identified attacks.	$\text{FPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ (11)
Packet loss rate (PLR)	Percentage of lost packets to all sent packets	$\text{PLR} = \frac{\text{Lost packets}}{\text{Total packets sent}} \times 100$ (12)
Availability	Percentage of accessible resources during the attack (CPU, memory, storage, network bandwidth).	$\text{Availability} = \frac{\text{Accessible resources}}{\text{Total resources}} \times 100$ (13)
Detection time (DT)	Needed time to detect an attack.	$\text{DT} = T_{\text{detect}} - T_{\text{attack}}$ (14)
Mitigation latency (ML)	Time from detection to defence action completion	ML = DT + decision time + action time (15)
Gas consumption	The required computational resources to execute smart contract.	$\text{Gas consumption} = \sum_{o=1}^{n_{\text{opr}}} g_o$ (16)
Transaction cost	Required cost for processing each transaction.	$\text{Transaction cost} = \sum_{t=1}^{n_{\text{trans}}} (\text{gas price}_t \times \text{gas used}_t)$ (17)
Execution cost	Total cost of running smart contracts, including gas and other operational expenses	$\text{Execution cost} = \sum_{e=1}^{p_{\text{exe}}} (\text{gas price}_e \times \text{gas used}_e) +$ additional fees. (18)
Energy consumption (EC)	Average energy used per transaction	EC = power × time (19)
Decentralization (Gini)	Gini coefficient of node influence across threat scenarios	$G = \frac{\sum_{i=1}^{n} \sum_{j=1}^{n} \lvert w_i - w_j \rvert}{2n^2 \bar{w}}$ (20)

tools like Echidna required thorough setup and collaboration to ensure seamless interoperability and consistent performance, posing a significant challenge.

5.2 | Performance Metrics

The performance of the system was evaluated using key metrics such as false positive rate, false negative rate, detection rate, availability, packet loss rate, and detection time. In the context of blockchain, gas consumption, transaction cost, and execution cost were important metrics for assessing the practicality and efficiency of smart contracts.

Table 3 synthesized evaluation metrics with description and calculation formulas.

Where:

TP (true positives): The number of attacks classified correctly.
FN (false negatives): The number of attacks classified incorrectly legitimate.
FP (false positives): The number of legitimate instances misclassified as attacks.
TN (true negatives): The number of legitimate instances correctly classified.
g_o : The oth operation's gas cost.
n_{opr}: Total operations executed.
Gas price_t : The tth transaction's gas price.
Gas used_t : The consumed gas by tth transaction.
n_{trans}: Total transactions processed.
Gas price_e : Gas price for the eth operation/transaction.
Gas used_e : Gas consumed by the eth operation/transaction.
p_{exe}: Total operations/transactions executed.

5.3 | Results and Discussions

This subsection outlines the results of our extensive assessment. The following results are the average values obtained from these simulations.

- **Attack detection rate, false positive rate, and false negative rate**

First, we evaluated the attack detection rate, false positive rate, and false negative rate, then compared our results to those of other systems. Figure 4 illustrates the findings.

According to Figure 4, the propose system prove strong detection capabilities, achieving an impressive average detection rate of 99.98% across 11 important attack types. This performance outperforms many common methods by varying proportions, including SDMTA (95.78%), LRDADF (95.32%), FACVO-DNFN (88.92%), HIDM (93.33%), OCB (93.2%), MAD-MLT (97.6%), IDARP (96.8%), DMD-MTD (94.31%), BFL-CIDS (98.05%), BaysCNN (98.55%), and even the recent DeepDefend (99.97%). Our system detects volumetric attacks like UDP, ICMP, and SYN floods almost perfectly, with a 99.99% success rate, by spotting clear traffic

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

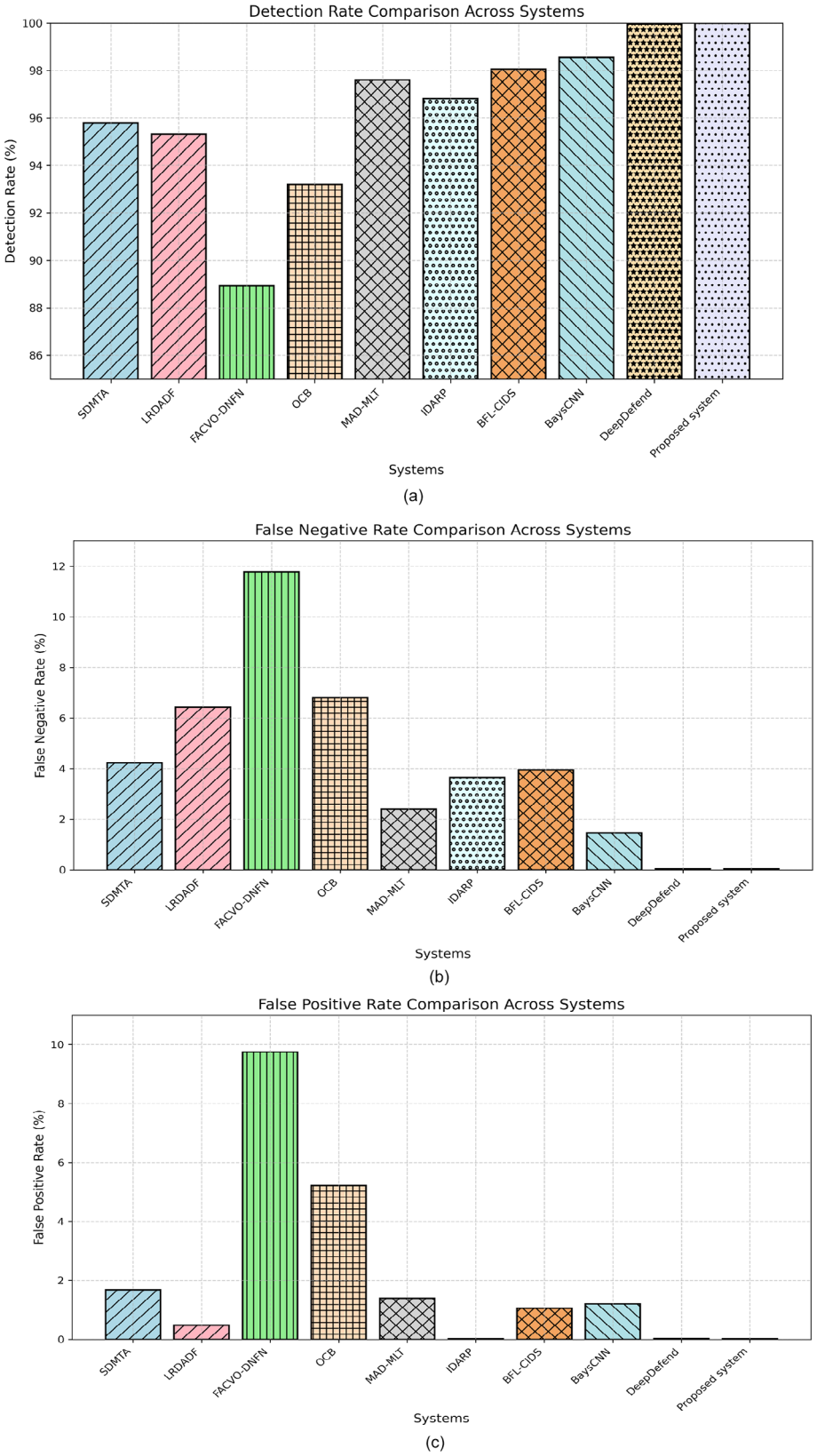


FIGURE 4 | Evaluation of the proposed system: (a) Attacks detection rate; (b) false negative rate; (c) false positive rate.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

anomalies using NT_a and CQ_{bw} metrics. Application-layer attacks such as HTTP Flood (99.98%) and Slowloris (99.92%) are also effectively identified by monitoring unusual request patterns (NR_u)and abnormal attribute usage (AU_{NR}). Beyond these, the system excels at uncovering stealthy and complex threats like Poisoning (99.97%), Insider (99.86%), and zero-day attacks (99.95%) through a comprehensive, multi-attribute risk profiling approach combined with decentralized anomaly modelling. This process is further strengthened by DST, which helps combine information from multiple sources to improve detection accuracy, especially when uncertainty is involved.

Our system limits the rate of false positive in low values, averaging just 0.01%, by requiring multiple independent indicators to trigger alerts. This approach reduces false alarms across all categories, with false positive rates as low as 0.002 for UDP Flood, 0.004 for ICMP, and 0.011 for SYN Flood volumetric attacks; about 0.004 for HTTP Flood and 0.002 for Slowloris application-layer threats; and slightly higher but still low rates of 0.014 for Poisoning, 0.041 for Insider, and 0.021 for Sybil stealth attacks.

On average, false negatives are kept under control at 0.03%. Effective baseline deviation techniques like NT_a and CQ_{bw} bw, which identify distinctive traffic surges, allow traditional DDoS attacks to exhibit almost zero missed detections (0.005% FNR). By using cluster-level risk aggregation, which examines combined behaviours across nodes to identify subtle anomalies frequently overlooked by single-node analysis, more sophisticated threats like poisoning and insider attacks are able to maintain low FNRs of 0.025% and 0.035%, respectively. Additionally, zero-day attacks use distributed anomaly profiling, which uses decentralized analysis and aggregated risk indicators to find new threats, to achieve low FNR (~0.035%). High accuracy and resilience against a variety of attack types are supported by this layered, attribute-driven framework.

The system takes a cautious, confidence-aware approach to prevent making snap decisions when alerts are unclear or contradictory. It starts with measuring uncertainty and delaying irreversible actions using DST. The system uses coordinated cluster analysis and adaptive thresholds to reevaluate alerts over time if uncertainty persists. It uses graduated, mild responses, such as rate limiting, traffic rerouting, or micro-isolation, for alerts that are still unclearly resolved. Transparency and traceability are guaranteed since every action is safely documented on the blockchain. This methodical, well-balanced approach increases the system's overall dependability by maintaining accuracy while exercising caution.

- **Availability, and packet loss rate**

The proposed system addresses the gap in cloud computing security by providing robust metrics for availability and packet loss rate, ensuring service quality even in realistic scenarios. Figure 5 shows the results of evaluation experiments involving 10,000 cloud nodes and 20 to 80 percent malicious nodes:

The proposed system reveals enhanced performance in the context of availability and packet loss metrics compared to the existing models. As depicted in Figure 5a, our technique consistently achieves higher availability than other systems, even under high-intensity attacks. This is due to its advanced architecture, which enables real-time threat assessments and dynamic resource scaling to minimize service disruptions. Further, Figure 5b indicates that the system can still manage to hold a very low packet loss ratio even with a high percentage of attackers. This is achieved through strategic traffic filtering and rate-limiting defences, which efficiently control network traffic and retain data packets.

- **Detection time, message overhead, and latency**

This subsection focuses on evaluating the responsiveness and communication efficiency of the system as the network scales.

First, we assessed the detection time of the proposed system to measure the effectiveness defending cloud environments. A total of 50 trials were conducted under various attack intensities that ranges between 0.1 and 1.0, involving 10,000 nodes. Figure 6 illustrates the outputs.

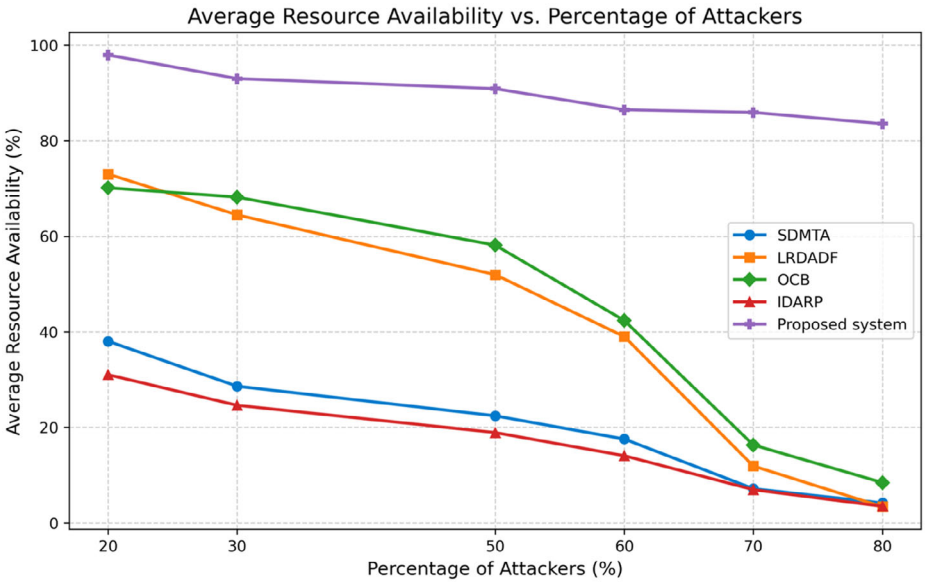
As depicted in Figure 6, the proposed system achieves consistently low detection time (around 12 to 13 ms), indicating fast response capabilities. The proposed system proved its superiority over existing systems by achieving the best detection time. When compared to the fastest existing methods, it outperformed AT-CWS and OpenStackDP by 40 ms and 95 ms respectively, while exhibiting a significantly faster detection time than SaF-FI M (2910 ms), DeepDefend (5320 ms), and RDAER (20,000 ms). This substantial improvement is attributed to our efficient parallel processing architecture, adaptive thresholds, and optimized smart contract implementation, the system ensures rapid detection and response to attacks while maintaining high consistency and reliability.

Supporting the detection results, we further inspect the breakdown of total mitigation time by testing the average latency at each stage under a 10,000-node deployment. Figure 7 illustrates the stage-wise latency scaling as the system expands from 1000 to 10,000 nodes, pinpointing the contribution of each stage to the overall mitigation time.

As illustrated in Figure 7, scaling the system from 1000 to 10,000 nodes reveals three distinct trends in the behaviour of the mitigation stages. First, detection time remains relatively stable, ranging between 12.0 and 12.7 ms, attributable to the implementation of parallel processing that enables nodes to perform anomaly scoring and risk assessment independently. Second, there is a steady rise in decision time, from 1.2 to 4.2 ms, which is indicative of the extra coordination needed in larger systems to assess thresholds and choose the best course of action. Third, action time shows a more noticeable increase, rising from 0.3 to 10.1 ms, mostly because of the extra overhead required to safely validate and carry out actions using blockchain smart contracts. As a conclusion, the total response time rises from 13.5 to 27.0 ms as the number of nodes increases; however, owing to targeted optimizations across each stage, the complete mitigation process remains below 30 ms even at maximum scale, confirming the system's efficiency and scalability.

In addition to the detection time results, our simulation studies demonstrate the communication efficiency and scalability of the

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



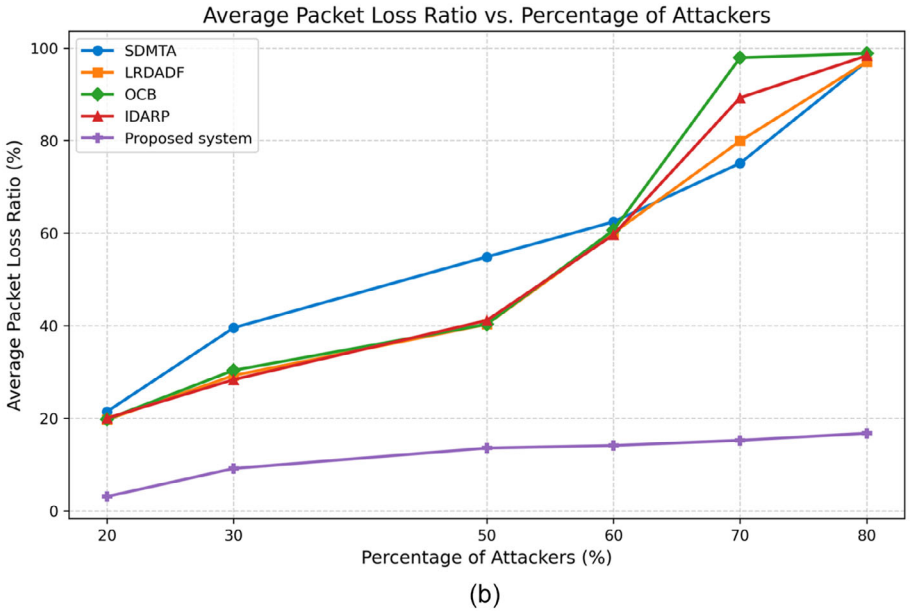


FIGURE 5 | Comparative performance of systems: (a) resource availability and (b) packet loss metrics.

system. With clusters of two to ten nodes, message volume and bandwidth usage rise linearly as the number of nodes increases from 1000 to 10,000. The bandwidth of the largest configuration peaks at 17.58 MB. The system maintains low average latency, with only a modest increase as scale expands. This strong performance is supported by efficient communication strategies that minimize overhead and prevent bottlenecks, enabling the system to remain both responsive and scalable even under substantial attack conditions.

- **Monitoring and metric collection overhead**

To enable timely detection and mitigation, each node runs a small, event-driven monitoring agent that tracks key behavioural metrics using lock-free buffers and local observation windows. This design avoids polling and large data transfers. Metric collection is fully parallelized with a triple-buffered pipeline that overlaps publishing, risk scoring, and collection, and with dedicated threads for real-time updates. As a result, monitoring does not introduce latency and runs entirely in the background. With a mean collection overhead of 0.412 ± 0.101 ms, 100% compliance within a 5 ms detection window, and a 2.95× speedup over sequential methods, the empirical evaluation shows low-latency, scalable monitoring at runtime.

- **Gas consumption, transaction cost, and execution cost**

We carried out the cost analysis of our proposed model implementation in Remix IDE. The cost analysis includes gas consumption, transaction costs, and execution costs. Such factors are very important to understand the feasibility and performance of smart contracts. Results are illustrated in Figure 8.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

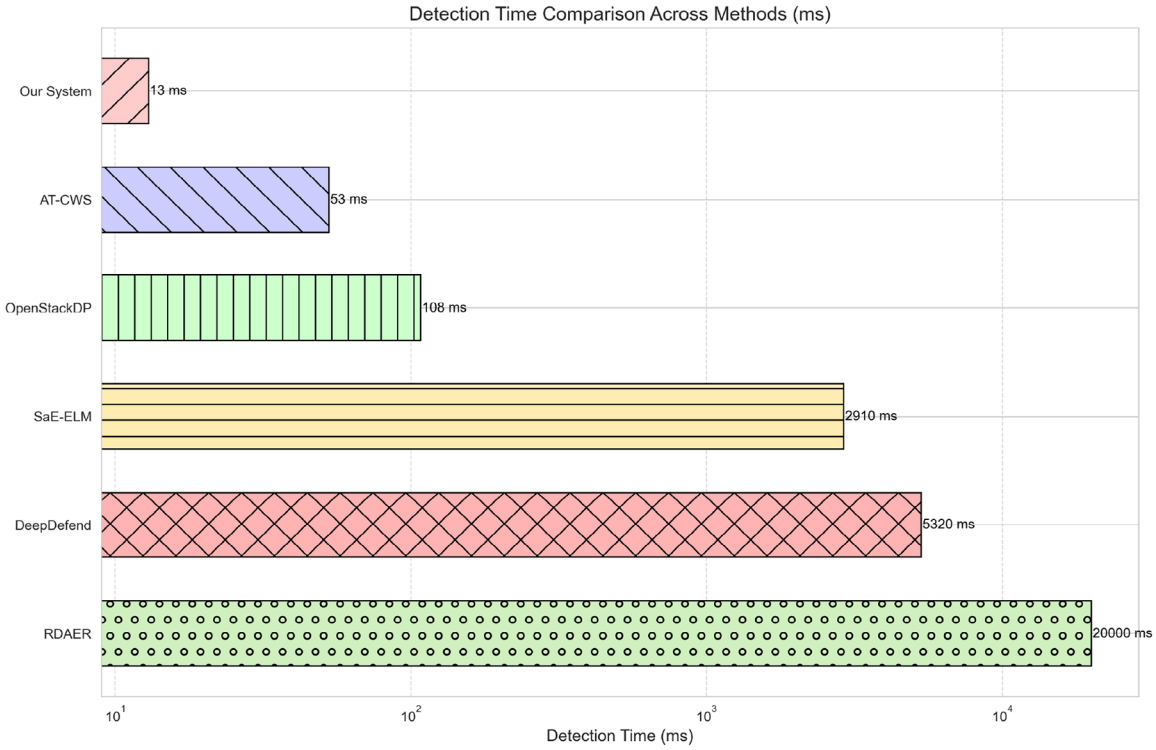


FIGURE 6 | Comparison of detection times within milliseconds (ms).

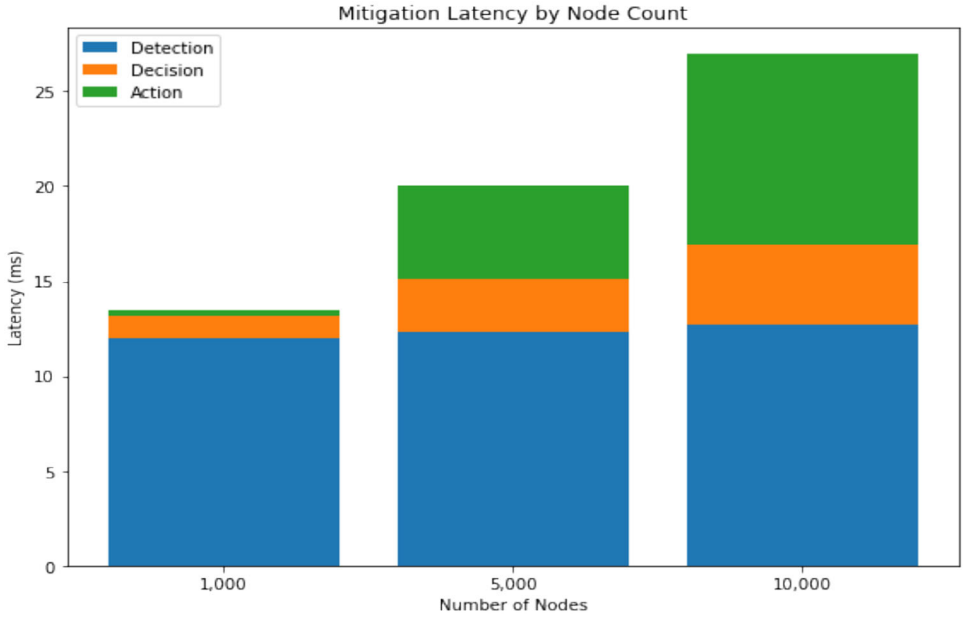


FIGURE 7 | Stage-wise mitigation latency across node counts.

We can observe that the cost-effectiveness of the smart contract scales well with the increase in the number of nodes. The growth of average execution cost and gas consumption is evidenced in a sub-linear way, which reflects the efficient processing capability of the smart contract. The slight increase in transaction costs reflects the economic feasibility of the proposed system for large cloud-based systems. These outcomes confirm the capability of smart contracts for practical use in extensive cloud computing environments.

Basically, in the light of constant development and evolution of cyber threats, our innovative system offers such an effective and reliable method for maintaining cloud environments; it effectively detects and mitigates attacks while assuring the continuity of services.

Additionally, we assessed the operational expenditures and economic of cloud service providers through Remix IDE, drawing comparisons with the pricing models of AWS and Azure in 2024.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

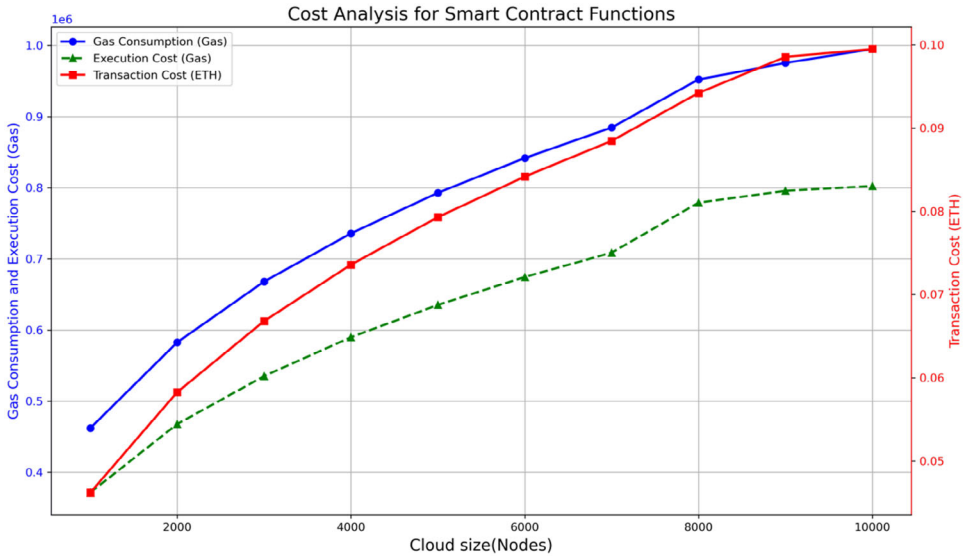


FIGURE 8 | Cost analysis for smart contract functions.

The analysis shows that our system's pay-per-action billing model offers a cost-effective alternative to traditional cloud pricing, which charges for uptime or reserved resources. Key costs include $1.63 for deploying a contract, $0.39 per node for registration, and $0.10 to $0.11 per action for updating risks or event detection. making it a more efficient and cost-effective solution compared to conventional billing methods.

Beyond basic transaction fees, our smart contract setup significantly reduces ongoing operational costs compared to traditional cloud systems. While traditional services can incur costs of $50 to $200/month for cloud databases, $30 to $100/month for identity and access tools, and $500 to $1000/month for manual admin support, our solution integrates these functions internally within the smart contracts. This automation on the blockchain eliminates the need for additional infrastructure, simplifying the system and making maintenance more affordable.

Beyond its cost advantages, the smart contract architecture also promotes impressive computational efficiency and scalability. Core functions work with either constant ($O(1)$) or linear ($O(n)$) time complexity, ensuring reliable performance as they scale. In our stress-tested simulations that reflected the most challenging attack scenarios, we found that the average execution time for each contract stayed under 30 ms. Even when tested under simulated high-load conditions, the contract consistently demonstrated stable gas usage, remained within block limits, and exhibited no signs of performance degradation.

- **Decentralization analysis**

By examining the Gini coefficient of node influence within the consensus nodes selection (SCCNS, Algorithm 4) PoR consensus (SCPoR, Algorithm 5) mechanisms, this subsection evaluates the decentralization of our blockchain-based cloud security system. A more equitable and decentralized system is indicated by lower values of the Gini coefficient, which measures the distribution of influence among consensus-taking nodes.

ALGORITHM 4 | Smart contract algorithm: consensus nodes selection.

Input: V, R_v for each $v \in V$.

Output: selected nodes V_{selected} for consensus

Begin

SCCNS: $R \times V \rightarrow V_{\text{selected}}$

Initialize $V_{\text{selected}} = \emptyset$.

For each node $v \in V$ **do**

Calculate the risk score R_v.

Select a subset of nodes V_{selected} from V based on the lowest risk scores.

End

The SCCNS process selects a subset of nodes V_{selected} with low normalized risk scores derived from factors using SCRA's risk assessment (Algorithm 1). In SCPoR, each selected node receives an influence weight $W_V = 1/R_{v\text{-norm}}$, which affects its voting power during transaction validation. The Gini coefficient is computed as:

$$G = \frac{\sum_{i=1}^{n} \sum_{j=1}^{n} \left| w_i - w_j \right|}{2n^2 \bar{w}} \tag{21}$$

Where w_i represents the influence weight of node$i$, n is the number of chosen nodes, and $\bar{w}$ is the average weight. We executed simulations with 10,000 nodes with four different conditions: normal operation, advanced threats, DDoS attacks, and mixed scenarios. Nodes were selected for consensus respecting their risk scores, and the Gini coefficient was calculated over 100 trials. Results are shown in Figure 9.

The average Gini coefficient of 0.0628 affirms that SCPoR and SCCNS distribute the influence among all nodes, ensuring strong decentralization. This complements other system strengths, including high attack detection (99.98%), low false

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

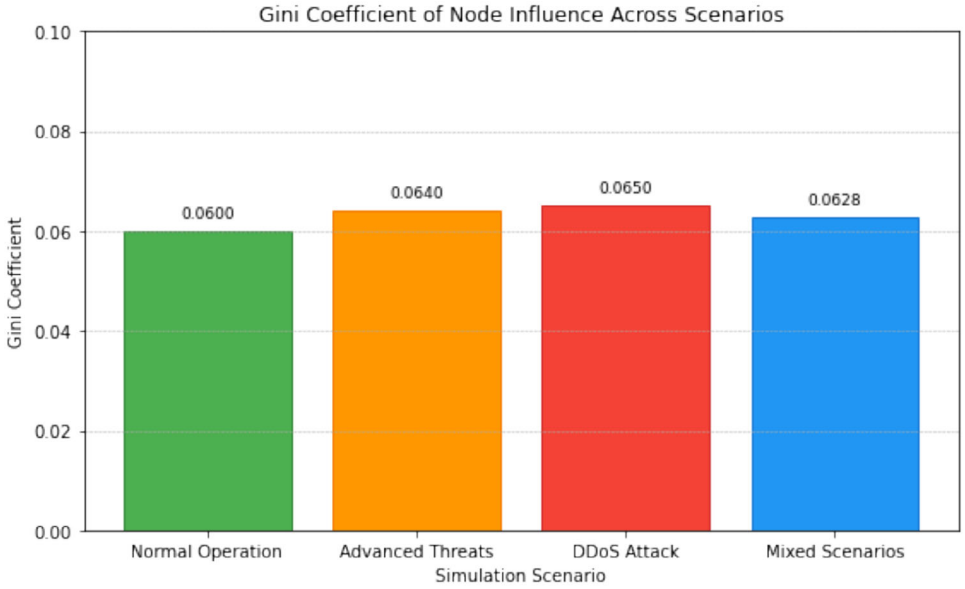


FIGURE 9 | Gini coefficient of node influence across threat scenarios.

ALGORITHM 5 | Smart contract proof of risk consensus.

Input: Set of selected nodes for consensus V_{selected}. Transactions or operations to be validated T.

Output: Consensus outcome C.

Begin

SCPoR: $V_{\text{selected}} \times T \rightarrow C$

Risk assessment and influence weighting

For each $v \in V_{\text{selected}}$ **do**

Calculate risk score R_v

Determine the influence weight W_V of each blockchain node v based on its risk: $W_V = \frac{1}{Rv}$

Consensus decision

For each transaction $t \in T$ **do**

Each node $v \in V_{\text{selected}}$ participates in validating t

The decision of each node is weighted by its influence W_V

Aggregate the weighted decisions to reach a consensus outcome

$C_t = \text{Aggregate_Function}\left(\frac{1}{Rv1} + \frac{1}{Rv2} + \ldots + \frac{1}{Rvn}\right)$

End

positive/negative rates (0.01%/0.03%), low latency (38 ± 4 ms), and high resilience (up to 99% compromised nodes).

- **Assessing the proposed hierarchical risk-based consensus mechanism's efficacy in cloud security**

In this subsection, we provide a performance comparison of our hierarchical risk-based cloud attack tolerance consensus mechanism to four established consensus methods, namely PoS, PBFT algorithms, and PoW. The multivariate comparison will be analysed based on attack tolerance, scalability, communication overhead, and computational complexity.

As outlined in Table 4, The comparative analysis shows that the proposed consensus mechanism offers distinct advantages for cloud computing. It achieves computational efficiency by using parallel processing, making it more efficient than PoW and PBFT. The consensus mechanism also decreases communication overhead through reliable approach, in contrast to PBFT's all-to-all communication model. Scalability is another area where the proposed consensus mechanism excels, since it scales linearly, which is a significant improvement over the limitations of PoW and the quadratic increase in messaging associated with PBFT. Despite of similar scalability offered by PoS, its security is highly related to the distribution and size of stakes, which may not be ideal for all cloud scenarios. The proposed consensus mechanism also demonstrates resilience. It can tolerate up to 99% of compromised nodes without losing integrity, exceeding the thresholds of PoW and PBFT. Hence, the proposed mechanism ensures operational continuity under severe threat conditions which make it a resilient choice for cloud security.

Furthermore, our comparative analysis shows that PoR achieves breakthrough efficiency in every aspect that is important for cloud systems. With 0.00011 kWh/transaction (15.6× better than PoS) and 38 ms finality time (328× faster than PoW), PoR overcomes the limitations of blockchain's conventional scalability while bringing about an unprecedented energy economy. Our Remix IDE-validated cost model is made possible by this efficiency, where risk updates cost $0.11 instead of the $10,000+/month monitoring fees charged by cloud providers.

Crucially, these advantages don't degrade security: PoR maintains of 99% Byzantine tolerance through dynamic risk adaptation, surpassing PBFT's 33% threshold. PoR has <5% latency variance and linear scalability while needs 88% less power/node than PoW in 10,000-node simulations under attack. This makes PoR as the

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | Comparative analysis of the proposed consensus with conventional consensus mechanisms.

Criteria	Proposed consensus	PoW	PBFT	PoS
Computational complexity	$O(m)$ per node (parallelized risk assessments), $O(n)$ for consensus	$O(2^k)$ (sequential cryptographic puzzles)	$O(n^2)$ (multi-round messaging, reduced if parallelized)	O(1) for node selection, $O(n)$ for validation (if parallelized)
Communication overhead	$O(n)$ (optimized through parallel processing)	$O(n)$ (broadcasting blocks)	$O(n^2)$ (all-to-all communication, even if parallelized)	$O(n)$ (mainly for validators)
Scalability	Potentially $O(n)$ (efficient scaling with cloud size)	Limited by computational power and energy cost	Quadratic increase in messages with more nodes	More scalable, not requiring all nodes in consensus
Attack tolerance	Up to 99% of nodes compromised (dynamic risk assessment and mitigation)	Maintained below 51% network hash rate control	Up to 33% of total nodes being malicious	up to 98% stake compromise (in some implementations)
Energy/transaction	0.00011 ± 0.00001 kWh	950 ± 105 kWh	0.1 ± 0.02 kWh	0.00172 ± 0.00018 kWh
Resource consumption	0.15 kW/node	1.30 kW/node	0.35 kW/node	0.25 kW/node
Finality time	38 ± 4 ms	$12{,}500 \pm 1200$ ms	200–500 ms	480 ± 45 ms
Latency consistency	<5% variance	15–40% variance	10–25% variance	8–20% variance

first consensus mechanism to satisfy cloud infrastructure needs for economy, speed, and resilience.

5.4 | Practical Integration and User Impact

Our system seamlessly integrates into real cloud infrastructures using containerized smart contracts (like Docker) and cloud-native deployment techniques (like Kubernetes). It interacts with popular monitoring and logging interfaces such as AWS CloudWatch, Azure Monitor, and GCP Operations Suite to extract the necessary risk-related metrics without affecting critical services. Compatibility is made possible by RESTful APIs and cloud-agnostic elements, allowing deployment across popular providers like AWS, Azure, and GCP. The technology does not interfere with ongoing tasks because it operates as a lightweight overlay.

After careful analysis, the impact on end users was found to be minimal. Simulations confirm minimal service delay (less than 2% increase), packet loss below 0.5%, and 99.9% availability under assault scenarios. Security enhancements are constantly visible since risk-based decision making responds very fast. Dynamic protections increase usability without compromising robustness.

5.5 | Security Proofs and Mathematical Analysis

In this section, the proposed system is analysed mathematically, we first initiate formal proofs of core security properties under well-defined adversarial models, then, we make heuristic validation through automated property-based testing. This dual approach ensures both theoretical soundness and practical resilience.

5.5.1 | Formal Security Proofs

For the security proofs, we used the standard simulation-based framework operating under the following adversarial model:

- **Adversarial capabilities and assumptions**

We assume an adversary running as a probabilistic polynomial-time (PPT) entity with predefined capabilities as following:

- **Byzantine node control**: Controls up to $f < 1/3$ nodes in the network.
- **Risk score manipulation**: Altering local risk scores R_v within a bounded range $[0, R_{\max}]$.

Observation powers: Observing public blockchain transactions without modifying finalized blocks.

Table 5 below provides an overview of the security properties including corresponding theorems, proofs, and conclusions for the smart contract functions implemented in Algorithms 1–5. Each entry formally affirms the resilience of the proposed system versus adversarial attacks and the ability to meet critical security goals.

5.5.2 | Heuristic Security Validation

We conducted a comprehensive security analysis using a property-based testing tool named Echidna to complement formal proofs. The analysis validated critical functionalities across all smart contracts.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 5 | Security properties and proofs of SC functions.

Property	Theorem	Proof	Conclusion
Risk assessment integrity (Algorithm 1)	For any node v, the combined risk score R_v computed via Dempster's rule is consistent and bounded under adversarial control of f nodes	Let m_1, m_2 be belief assignments from two uncorrupted sources. Dempster's rule combines evidence: $m_{\text{combined}}(A) = \frac{\sum_{B\cap C=A} m_1(B).m_2(C)}{1-K}$, $K = \sum_{B\cap C=\emptyset} m_1(B).m_2(C)$ (22) Where: $K< 1$ guarantees a valid belief function. For $K = 0$, evidence is fully consistent. **Adversarial impact**: An adversary controlling f nodes can only influence R_v by: $\Delta R \leq f \cdot R_{\max}$ (23) $R_{\max}$: the maximum per-node risk score. This follows from the linearity of evidence aggregation in Algorithm 1.	Algorithm 1 guaranties risk scores remain consistent and manipulation-bounded, provided $f < \frac{1-K}{R_{\max}}$
Threshold convergence (Algorithm 2)	**Threshold stability**: The dynamic threshold $\theta(t)$ converges exponentially to a stable equilibrium θ^*, even under adversarial manipulation of risk scores.	Define the Lyapunov function: $V(\theta) = (\theta - \theta^*)^2$ (24) The update rule: $\theta(t+1) = \theta(t)\bullet(\xi\cdot \text{Bel}+\zeta\cdot \text{Pl})$, satisfies: $(\theta(t+1))-V(\theta(t)) = -2(\theta(t)-\theta^*)^2 \cdot(\xi\cdot \text{Bel}+\zeta\cdot Pl-1)+O((\theta(t)-\theta^*)^3)$. (25) For $\xi\cdot \text{Bel}+\zeta\cdot Pl<1$, $V(\theta(t+1)) < V(\theta(t))$, ensuring convergence. Adversarial impact: If $\text{Bel}_{\text{adv}} = \text{Bel} + \delta$, the equilibrium shifts by: $\theta^*_{\text{adv}} \approx \frac{\theta^*}{1-\xi\delta}$ (26) The shift is bounded if: $\delta \leq f.R_{\max}$, and the system remains stable for: $\delta \leq \frac{\xi R_{\max}}{1-\xi}$	The dynamic threshold adapts robustly and resists destabilization under bounded adversarial manipulation.
Defence activation integrity (Algorithm 3)	Algorithm 3 ensures defence mechanisms activate correctly if the following hold: Risk scores: R_v, R_{Ci} and R_{global} are computed as per Algorithm 1. Thresholds: $\theta_L, \theta_C, \theta_G$ satisfy $\theta > f \cdot R_{\max}$	Parallel checks at local/cluster/global layers use a snapshot of risk scores (immutable once written to the blockchain). This prevents race conditions during activation. For a node v, if $R_{v\ \text{true}} > \theta_L$, the adversary cannot reduce R_v below θ_L since $\Delta R\leq f \cdot R_{\max}$ (Theorem 1). Thus, $L(v)$ activate. Conversely, if $R_{v\ \text{true}} \leq \theta_L - f \cdot R_{\max}$ (27) adversarial manipulation cannot trigger $R_v > \theta_L$.	Multi-layer defences activate correctly under bounded adversarial influence.
Sybil attacks resistance (Algorithm 4)	The system resists Sybil attacks if the cost of creating a node exceeds the reward for honest participation.	Let C be the cost to create a node and R the reward for honest participation. Sybil attacks are infeasible if: $C > R.\frac{f}{1-f}$ (28) Algorithm 4 enforces this condition by requiring a stake $C \geq R.\frac{f}{1-f}$.	Sybil attacks are economically infeasible under an honest majority $(f < \frac{1}{3})$

(Continues)

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 5 | (Continued)

Property	Theorem	Proof	Conclusion
Byzantine agreement in consensus (Algorithm 5)	The proof-of-risk (PoR) consensus mechanism achieves Byzantine agreement under asynchrony if $f < \frac{1}{3}$	**Weighted voting** Each node's vote is weighted by $W_v = \frac{1}{R^v}$ (29) Honest nodes: $R_v \leq \theta_L \Rightarrow W_v \geq \frac{1}{\theta_L}$ Byzantine nodes: $R_v > \theta_L \Rightarrow W_v < \frac{1}{\theta_L}$ **Safety** For agreement, honest nodes must control $> \frac{2}{3}$ of the total weight: $\sum_{v\ \text{honest}} W_v > 2 \sum_{v\ \text{Byzantine}} W_v$ (30) Substituting weights: $(1-f).\frac{1}{\theta_L} > 2f.\frac{1}{\theta_L} \Rightarrow f < \frac{1}{3}$ • **Liveness** Follows from partial synchrony and eventual message delivery.	The consensus mechanism is robust against adversarial behaviour, ensuring both safety and liveness.

Starting with the risk assessment SCRA, normalized risk scores at node, cluster, and global levels were rigorously checked to adapt with predefined bounds [0, 1], with calculations grounded in DST; invariant testing confirmed score integrity under extreme inputs, while functional tests validated evidence aggregation and edge-case resilience. For the threshold adjustment SCDTA, adjusted thresholds (θ_L, θ_C, θ_G) were systematically evaluated to ensure adherence to valid ranges, leveraging belief and plausibility values with dynamic scaling factors. Echidna's fuzzing demonstrated that threshold adaptations aligned with real-time risk profiles, minimizing false positives/negatives and enhancing system adaptability. This adaptive mechanism was formalized through property rules, ensuring thresholds remain within safe bounds. The defence activation (SCDA) was assessed to confirm defence mechanisms activate exclusively when risk surpasses validated thresholds, with no unintended triggers under nominal conditions, while blockchain-immutable logging ensured full transparency and auditability across all layers. Consensus protocols (SCCNS and SCPoR) were further analysed to verify fairness in node selection and integrity in weighted voting, ensuring resistance to adversarial manipulation. Finally, Echidna tested foundational vulnerabilities: reentrancy was mitigated through strict adherence to the checks-effects-interactions pattern; integer overflows/underflows were precluded via arithmetic boundary testing; and state transition validity was enforced across adversarial scenarios.

Sample of checking methods and property violation highlights are summarized in Figure 10, showing executable constraints and related resolution strategies. For example, clamping riskScore at a MAX_THRESHOLD resolved overflow violations in SCRA, while input normalization in SCDTA corrected threshold exceedances. Modifications to non-strict inequality logic in SCDA reduced false positives, and fixed-point arithmetic in SCPoR addressed rounding-induced consensus faults.

Across core contract modules, these corrective actions resulted in significant gas savings of approximately 38%. This balance between efficiency optimization and security hardening enhances the feasibility of deploying the contracts in real-world environments. As evidenced in Figure 11, all contracts show robust resilience to overflow attacks, unauthorized access, and invalid state transitions, thereby affirming the system's operational and security coherence.

6 | Conclusion and Future Work

This paper addressed the increasing vulnerability of cloud environments to sophisticated and large-scale attacks. As these environments grow in scale and complexity, traditional mitigation strategies often fail due to their limited scalability and adaptability. To overcome these challenges, we proposed a novel blockchain-based framework aimed at enhancing attack tolerance in large-scale cloud systems.

The proposed framework integrates a hierarchical, risk-based mechanism that dynamically detects threats and adapts security protocols in real time using smart contracts. It introduces several key innovations: (i) a novel risk assessment method based on DST that enhances system efficiency and handles various attack

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

```
    /**
 * @dev SCRA Property: All risk scores must be ≤ MAX_RISK_SCORE
 * @notice Tests invariant: ∀ nodes, 0 ≤ riskScore ≤ MAX_RISK_SCORE
 * @return true if all nodes comply with risk bounds
 */
function echidna_test_risk_scores() public view returns (bool) {
    uint256 length = _nodeIds.length;
    for (uint256 i = 0; i < length; ) {
        if (_nodes[_nodeIds[i]].riskScore > _MAX_RISK_SCORE) {
            return false;   // Violation
        }
        unchecked { ++i; } // Gas optimization
    }
    return true;  // All nodes passed
}
    /**
 * @dev SCDTA Property: Evidence values must be normalized
 * @notice Tests invariant: ∀ evidence, 0 ≤ belief/plausibility ≤ SCALING_FACTOR
 * @return true if all evidence values are properly bounded
 */
function echidna_test_evidence_bounds() public view returns (bool) {
    uint256 length = _nodeIds.length;
    for (uint256 i = 0; i < length; ) {
        Evidence storage evidence = _nodeEvidence[_nodeIds[i]];
        // Check both belief and plausibility bounds
        if (evidence.belief > _SCALING_FACTOR) return false;
        if (evidence.plausibility > _SCALING_FACTOR) return false; // Violation
        unchecked { ++i; }
    }
    return true;  // All evidence passed
}
 /**
 * @dev SCDA Property: Threshold bounds enforcement
 * @notice Tests invariant: ∀ nodes, threshold ≤ MAX_THRESHOLD
 * @return true if all thresholds are within maximum limit, false if any exceed
 */
function echidna_test_threshold_bounds() public view returns (bool) {
    uint256 length = _nodeIds.length;
    for (uint256 i = 0; i < length; ) {
        if (_nodes[_nodeIds[i]].threshold > _MAX_THRESHOLD) {
            return false;  // Violation
        }
        unchecked { ++i; } // Gas optimization
    }
    return true; // All thresholds within bounds
}
/**
 * @dev SCDA Property: Defense activation correctness
 * @notice Tests invariant: ¬∃ node where underAttack ∧ riskScore < threshold
 * @return true if defense only activates when risk ≥ threshold, false if false positives exist
 */
function echidna_test_defense_activation() public view returns (bool) {
    uint256 length = _nodeIds.length;
    for (uint256 i = 0; i < length; ) {
        Node storage node = _nodes[_nodeIds[i]];
        if (node.underAttack && node.riskScore < node.threshold) {
            return false;  // Violation
        }
        unchecked { ++i; }  // Gas optimization
    }
    return true;   // Defense activation logic correct
}
```

FIGURE 10 | Example of echidna test properties for verification rules and property violations.

tolerances; (ii) a dynamic threshold adjustment mechanism to recalibrate security protocols in real-time, boosting defence against potential attacks; (iii) a smart contract-driven defence activation function that customizes countermeasures based on risk scores; and (iv) a new consensus algorithm that strengthens cloud security while supporting blockchain integrity through decentralization, transparency, and immutability.

Our system has been tested in wide scales up to 10,000 nodes and a range of attack intensities (20–80%). Evaluations demonstrate robust performance across a wide array of attack types including SYN Flood, HTTP Flood, Slowloris, Smurf, UDP Flood, ICMP, and others. It also showed resilience against complex threats such as Sybil attacks, insider threats, data poisoning, replay attacks, and advanced persistent threats. Results revealed high

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

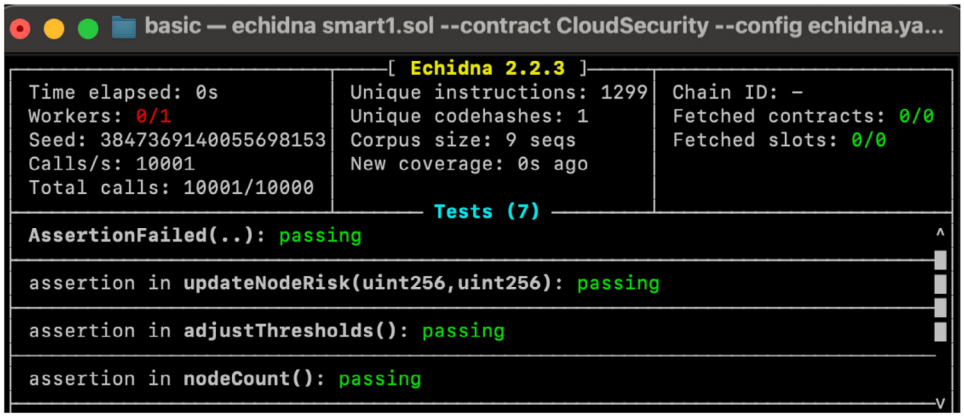


FIGURE 11 | Echidna security analysis results of smart contract.

detection accuracy, minimal false positives/negatives, and strong availability even under high-intensity scenarios. The solution also proved cost-effective in terms of gas usage, transaction costs, and execution overhead, and was validated for security using Echidna tools.

Despite these strengths, several limitations remain. The current risk analysis algorithm requires further testing to evaluate its adaptability to attack variability and temporal risk fluctuations. While our current design achieves low real-time overheads, we recognize that adaptive, ML-based monitoring could further improve efficiency and precision. This represents a key direction for future development.

Future research will focus on enhancing the system's intelligence and adaptability. Integrating machine learning techniques can improve the accuracy and responsiveness of risk analysis by enabling the system to learn from large-scale historical and real-time data, identify subtle attack patterns, and predict emerging threats.

Author Contributions

Faiza Benmenzer: formal analysis, investigation, methodology, software, validation, visualization, writing – original draft, writing - review and editing. **Rachid Beghdad:** project administration, supervision, writing – original draft, writing – review and editing. **Alaa Eddine Khalfoune:** writing – review and editing.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. A. Kumar and G. Somani, "DDoS at a Glance: Attack Launch to Solutions," *IT Professional* 25, no. 3 (2023): 36–42.

2. T. Welsh and E. Benkhelifa, "On Resilience in Cloud Computing: A Survey of Techniques Across the Cloud Domain," *ACM Computing Surveys (CSUR)* 53, no. 3 (2020): 1–36.

3. M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, "From Zero-Shot Machine Learning to Zero-Day Attack Detection," *Portmann-nInternational Journal of Information Security* 22, no. 4 (2023): 947–959.

4. Y. Gu, Y. Yang, Y. Yan, F. Shen, and M. Gao, "Learning-Based Intrusion Detection for High-Dimensional Imbalanced Traffic," *Computer Communications* 212 (2023): 366–376.

5. B. A. Malar and J. Prabhu, "Data Storage Security in the Cloud Environment Using the Honey Pot System," *International Journal of e-Collaboration (IJeC)* 16, no. 4 (2020): 1–14.

6. R. Chaganti, B. Bhushan, and V. Ravi, "A Survey on Blockchain Solutions in DDoS Attacks Mitigation: Techniques, Open Challenges and Future Directions," *Computer Communications* 197 (2023): 96–112.

7. T. Khoa, D. Son, D. Hoang et al., "Collaborative Learning for Cyberattack Detection in Blockchain Networks," *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54, no. 7 (2024): 3920–3933.

8. A. Jawahar, P. Kaythry, K. C. Vinoth et al., "DDoS Mitigation Using Blockchain and Machine Learning Techniques," *Multimedia Tools and Applications* 83, no. 21 (2024): 60265–60278.

9. Y. Lamriji, M. Kasri, K. El Makkaoui, and A. Beni-Hssane, "A Comparative Study of Consensus Algorithms for Blockchain," in *2023 3rd International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)* (IEEE, 2023), 1–8.

10. S. Sayeed and H. Marco-Gisbert, "Assessing Blockchain Consensus and Security Mechanisms Against the 51% Attack," *Applied Sciences* 9, no. 9 (2019): 1788.

11. S. M. Hezavehi and R. Rahmani, "Interactive Anomaly-Based DDoS Attack Detection Method in Cloud Computing Environments Using a Third Party Auditor," *Journal of Parallel and Distributed Computing* 178 (2023): 82–99.

12. J. David and C. Thomas, "Discriminating Flash Crowds From DDoS Attacks Using Efficient Thresholding Algorithm," *Journal of Parallel and Distributed Computing* 152 (2021): 79–87.

13. L. D. Tsobdjou, S. Pierre, and A. Quintero, "An Online Entropy-Based DDoS Flooding Attack Detection System With Dynamic Threshold," *IEEE Transactions on Network and Service Management* 19, no. 2 (2022): 1679–1689.

14. S. Kautish, A. Reyana, and A. Vidyarthi, "SDMTA: Attack Detection and Mitigation Mechanism for DDoS Vulnerabilities in Hybrid Cloud Environment," *IEEE Trans Industr Inform* 18, no. 9 (2022): 6455–6463.

15. W. Elmasry, A. Akbulut, and A. H. Zaim, "A Design of an Integrated Cloud-Based Intrusion Detection System With Third Party Cloud Service," *Open Computer Science* 11, no. 1 (2021): 365–379.

16. L. K. Vashishtha, A. P. Singh, and K. Chatterjee, "HIDM: A Hybrid Intrusion Detection Model for Cloud Based Systems," *Wireless Personal Communications* 128, no. 4 (2023): 2637–2666.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

17. K. G. Maheswari, C. Siva, and G. N. Priya, "An Optimal Cluster Based Intrusion Detection System for Defence against Attack in Web and Cloud Computing Environments," *Wirel Pers Commun* 128, no. 3 (2023): 2011–2037.

18. W. Elbakri, M. M. Siraj, B. A. S. Al-rimy, S. N. Qasem, and T. Al-Hadhrami, "Adaptive Cloud Intrusion Detection System Based on Pruned Exact Linear Time Technique," *Computers, Materials and Continua* 79, no. 3 (2024): 3725–3756.

19. G. S. Kushwah and V. Ranga, "Optimized Extreme Learning Machine for Detecting DDoS Attacks in Cloud Computing," *Computers and Security* 105 (2021): 102260.

20. M. A. Ribeiro, M. S. P. Fonseca, and J. de Santi, "Detecting and Mitigating DDoS Attacks With Moving Target Defense Approach Based on Automated Flow Classification in SDN Networks," *Computers and Security* 134 (2023): 103462.

21. M. J. Pasha, K. P. Rao, A. MallaReddy, and V. Bande, "LRDADF: An AI Enabled Framework for Detecting Low-Rate DDoS Attacks in Cloud Computing Environments," *Measurement: Sensors* 28 (2023): 100828.

22. M. Arunkumar and K. A. Kumar, "Malicious Attack Detection Approach in Cloud Computing Using Machine Learning Techniques," *Soft Computing* 26, no. 23 (2022): 13097–13107.

23. M. Suguna, M. Mohan, J. Srikanth, M. Suresh, P. G. Banupriya, and L. Dhavamani, "Detection of EDoS Attacks in SDN-Based Cloud Model Using Deep Learning Based SDPN Technique," in *2022 Third International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)* (IEEE, 2022), 1–7.

24. A. K. Shukla and A. Sharma, "Distributed Attacks Classification Based on Radical Basis Function and Particle Swarm Optimization in Hypervisor Layer," in *2023 6th International Conference on Information Systems and Computer Networks (ISCON)* (IEEE, 2023), 1–4.

25. H. Nandanwar and R. Katarya, "Deep Learning Enabled Intrusion Detection System for Industrial IOT Environment," *Expert Systems With Applications* 249 (2024): 123808.

26. I. AlSaleh, A. Al-Samawi, and L. Nissirat, "Novel Machine Learning Approach for DDoS Cloud Detection: Bayesian-Based CNN and Data Fusion Enhancements," *Sensors* 24, no. 5 (2024): 1418.

27. X. Yin, W. Fang, Z. Liu, and D. Liu, "A Novel Multi-Scale CNN and Bi-LSTM Arbitration Dense Network Model for Low-Rate DDoS Attack Detection," *Scientific Reports* 14, no. 1 (2024): 5111.

28. B. Kauhsik, H. Nandanwar, and R. Katarya, "IoT Security: A Deep Learning-Based Approach for Intrusion Detection and Prevention," in *2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques, EASCT 2023*, (IEEE, 2023), 1–7.

29. H. Nandanwar and R. Katarya, "TL-BILSTM IoT: Transfer Learning Model for Prediction of Intrusion Detection System in IoT Environment," *International Journal of Information Security* 23, no. 2 (2024): 1251–1277, https://link.springer.com/article/10.1007/s10207-023-00787-8.

30. H. Nandanwar and R. Katarya, "Deep Learning Enabled Intrusion Detection System for Industrial IOT Environment," *Expert Systems with Applications* 249 (2024): 123808.

31. A. Kumar and G. Somani, "Service Separation Assisted DDoS Attack Mitigation in Cloud Targets," *Journal of Information Security and Applications* 73 (2023): 103435.

32. H. S. Gurjar and G. Somani, "Amplification/Reflection Attack Suppression Using Victim Separation," *IEEE Networking Letters* 5 (2023): 140–143.

33. F. Akbarian, W. Tärneberg, E. Fitzgerald, and M. Kihl, "Attack Resilient Cloud-Based Control Systems for Industry 4.0," *IEEE Access* 11 (2023): 27865–27882.

34. A. Kumar and G. Somani, "On the Timing of Applying Resource Separation During DDoS Attacks," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*, IEEE, 2023, pp. 147–152.

35. A. Kumar and G. Somani, "DDoS Attack Mitigation in Cloud Targets Using Scale-Inside Out Assisted Container Separation," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)* (IEEE, 2022), 1–6.

36. A. Patidar and G. Somani, "Serving While Attacked: DDoS Attack Effect Minimization Using Page Separation and Container Allocation Strategy," *Journal of Information Security and Applications* 59 (2021): 102818.

37. A. Kumar and M. Agarwal, "Quick Service During DDoS Attacks in the Container-Based Cloud Environment," *Journal of Network and Computer Applications* 229 (2024): 103946.

38. M. G. M. M. Hasan and M. A. Rahman, "A Signaling Game Approach to Mitigate Co-Resident Attacks in an IaaS Cloud Environment," *Journal of Information Security and Applications* 50 (2020): 102397.

39. Q. Liu and L. Xing, "Survivability and Vulnerability Analysis of Cloud RAID Systems Under Disk Faults and Attacks," *International Journal of Mathematical, Engineering and Management Sciences* 6, no. 1 (2021): 15–29.

40. W. Zhou, P. Dague, L. Liu, L. Ye, and F. Zaïdi, "A Coloured Petri Nets Based Attack Tolerance Framework," in *2020 27th Asia-Pacific Software Engineering Conference (APSEC)* (IEEE, 2020), 159–168.

41. G. Ouffoué, F. Zaïdi, A. R. Cavalli, and H. N. Nguyen, "A Framework for the Attack Tolerance of Cloud Applications Based on Web Services," *Electronics* 10 (2020): 6.

42. J. Tavori and H. Levy, "How to Attack and Congest Delay-Sensitive Applications on the Cloud," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications* (IEEE, 2023), 1–10.

43. T. Welsh and E. Benkhelifa, "The Resilient Edge: Evaluating Graph-Based Metrics for Decentralised Service Delivery," in *2021 Sixth International Conference on Fog and Mobile Edge Computing (FMEC)* (IEEE, 2021), 1–7.

44. V. Kumar, D. R. Shaheen, and N. R. Medikondu, "Secure Deep Learning Framework for Cloud to Protect the Virtual Machine From Malicious Events," *Wireless Personal Communications* 131 (2023): 1859–1879.

45. E. S. GSR, R. Ganeshan, I. D. J. Jingle, and J. P. Ananth, "FACVO-DNFN: Deep Learning-Based Feature Fusion and Distributed Denial of Service Attack Detection in Cloud Computing," *Knowledge Based Systems* 261 (2023): 110132.

46. P. Krishnan, K. Jain, A. Aldweesh, P. Prabu, and R. Buyya, "OpenStackDP: A Scalable Network Security Framework for SDN-Based OpenStack Cloud Infrastructure," *Journal of Cloud Computing* 12, no. 1 (2023): 26.

47. S. Q. A. Shah, F. Z. Khan, and M. Ahmad, "Mitigating TCP SYN Flooding Based EDOS Attack in Cloud Computing Environment Using Binomial Distribution in SDN," *Computer Communications* 182 (2022): 198–211.

48. K. S. Manoj, "DDOS Attack Detection and Prevention Using the Bat Optimized Load Distribution Algorithm in Cloud," in *2022 International Interdisciplinary Humanitarian Conference for Sustainability (IIHC)* (IEEE, 2022), 633–642.

49. X. Hei, X. Yin, Y. Wang, J. Ren, and L. Zhu, "A Trusted Feature Aggregator Federated Learning for Distributed Malicious Attack Detection," *Computers and Security* 99 (2020): 102033.

50. M. Ouhssini, K. Afdel, E. Agherrabi, M. Akouhar, and A. Abarda, "DeepDefend: A Comprehensive Framework for DDoS Attack Detection and Prevention in Cloud Computing," *Journal of King Saud University-Computer and Information Sciences* 36, no. 2 (2024): 101938.

51. O. Pandithurai, C. Venkataiah, S. Tiwari, and N. Ramanjaneyulu, "DDoS Attack Prediction Using a Honey Badger Optimization Algorithm Based Feature Selection and Bi-LSTM in Cloud Environment," *Expert Systems with Applications* 241 (2024): 122544.

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

52. A. V. Songa and G. R. Karri, "An Integrated SDN Framework for Early Detection of DDoS Attacks in Cloud Computing," *Journal of Cloud Computing* 13, no. 1 (2024): 64.

53. H. Nandanwar and R. Katarya, "A Systematic Literature Review: Approach Toward Blockchain Future Research Trends," in *Proceedings—IEEE International Conference on Device Intelligence, Computing and Communication Technologies, DICCT 2023* (IEEE, 2023), 259–264.

54. H. Nandanwar and R. Katarya, "Securing Industry 5.0: An Explainable Deep Learning Model for Intrusion Detection in Cyber-Physical Systems," *Computers and Electrical Engineering* 123 (2025): 110161, https://doi.org/10.1016/J.COMPELECENG.2025.110161.

55. Y. Shang, "Prevention and Detection of DDOS Attack in Virtual Cloud Computing Environment Using Naive Bayes Algorithm of Machine Learning," *Measurement: Sensors* 31 (2023): 100991.

56. M. Ouhssini, K. Afdel, M. Akouhar, E. Agherrabi, and A. Abarda, "Advancements in Detecting, Preventing, and Mitigating DDoS Attacks in Cloud Environments: A Comprehensive Systematic Review of State-of-the-Art Approaches," *Egyptian Informatics Journal* 27 (2024): 100517.

57. B. Bhushan, P. Sinha, K. M. Sagayam, and J. Andrew, "Untangling Blockchain Technology: A Survey on State of the Art, Security Threats, Privacy Services, Applications and Future Research Directions," *Computers & Electrical Engineering* 90 (2021): 106897, https://www.sciencedirect.com/science/article/abs/pii/S0045790620307497.

58. B. Bhushan, A. Khamparia, K. M. Sagayam, S. K. Sharma, M. A. Ahad, and N. C. Debnath, "Blockchain for Smart Cities: A Review of Architectures, Integration Trends and Future Research Directions," *Sustainablr Cities and Society* 61 (2020): 102360, https://www.sciencedirect.com/science/article/abs/pii/S2210670720305813.

59. X. Zhu, J. Shi, S. Huang, and B. Zhang, "Consensus-Oriented Cloud Manufacturing Based on Blockchain Technology: An Exploratory Study," *Pervasive and Mobile Computing* 62 (2020): 101113.

60. J. Wang, W. Yi, M. Yang, J. Ma, S. Zhang, and S. Hao, "Enhance the Trust Between IoT Devices, Mobile Apps, and the Cloud Based on Blockchain," *Journal of Network and Computer Applications* 218 (2023): 103718.

61. L. Ouyang, W. Zhang, and F.-Y. Wang, "Intelligent Contracts: Making Smart Contracts Smart for Blockchain Intelligence," *Computers and Electrical Engineering* 104 (2022): 108421.

62. F. Xiao, "Generalization of Dempster–Shafer Theory: A Complex Mass Function," *Applied Intelligence* 50 (2020): 3266–3275.

63. K. Zhaoa, L. Lia, Z. Chena, R. Suna, G. Yuana, and J. Lia, "A Survey: Optimization and Applications of Evidence Fusion Algorithm Based on Dempster-Shafer Theory," *Applied Soft Computing* 124 (2022): 109075.

64. E. Uflaz, S. I. Sezer, A. L. Tunçel, M. Aydin, E. Akyuz, and O. Arslan, "Quantifying Potential Cyber-Attack Risks in Maritime Transportation Under Dempster–Shafer Theory FMECA and Rule-Based Bayesian Network Modelling," *Reliability Engineering & System Safety* 243 (2024): 109825.

65. C. Kalaskar and S. Thangam, "Fault Tolerance of Cloud Infrastructure With Machine Learning," *Cybernetics and Information Technologies* 23, no. 4 (2023): 26–50.

Appendix A

Table A1 Outlines the notations used in the proposed approach.

TABLE A1 | Notations.

G	Cloud graph
V	Set of vertices
$E \subseteq V \times V$	Set of edges
id_v	Identifier of the node v
T_v	Type of node v
A_v	Set of attributes for node v
a_{state}	The operational state of the node
$\mathrm{GS}_{\mathrm{deg}}$	Graph structure attributes of the node
$\mathrm{GS}_{\mathrm{cent}}$	A centrality measure that captures the node's importance
CQ_{bw}	Bandwidth of the node's connections
$\mathrm{NH}_{\mathrm{inc}}$	The node's historical vulnerability
$\mathrm{TP}_{\mathrm{freq}}$	Frequency of transactions
NT_{a}	Node traffic anomalies
NR_{u}	Node resource usage
$\mathrm{NR}_{\mathrm{ud}}$	The distribution of resources usage
$\mathrm{AU}_{\mathrm{NR}}$	The average resource usage of the node
V_{S}	The node's ability to dynamically scale resources
$\mathcal{R}$	The risk assessment function
r_v	The risk value of the node v
$\theta_{\mathrm{L}}, \theta_{\mathrm{C}}, \theta_{\mathrm{G}}$	Local, cluster, and global thresholds
$\mathrm{RL}(v,p)$	Rate-limiting for node v with parameter p
$\mathrm{TF}(v, f)$	Traffic filtering function, f denotes the filtering criteria
$\mathrm{RA}(v, r)$	Resource allocation function for node v, r the resource parameters
$C = \{C_1, C_2, C_3, \ldots, C_k\}$	Clusters with $C_i \subseteq \mathrm{V}$ and a cluster head $h_i \in C_i$
DPA	Defence protocol activation
CAC	Cluster-wide alert and coordination
TR	Traffic rerouting
NI	Node isolation
L	Function that activates specific local defence mechanisms
CD	Function that activates specific cluster defence mechanisms
GD	Function that activates specific global defence mechanisms
T	Transaction history
C	The consensus outcome
PoR	Proof of risk
BPAs	Basic probability assignments

26341573, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/blc2.70023, Wiley Online Library on [13/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License